

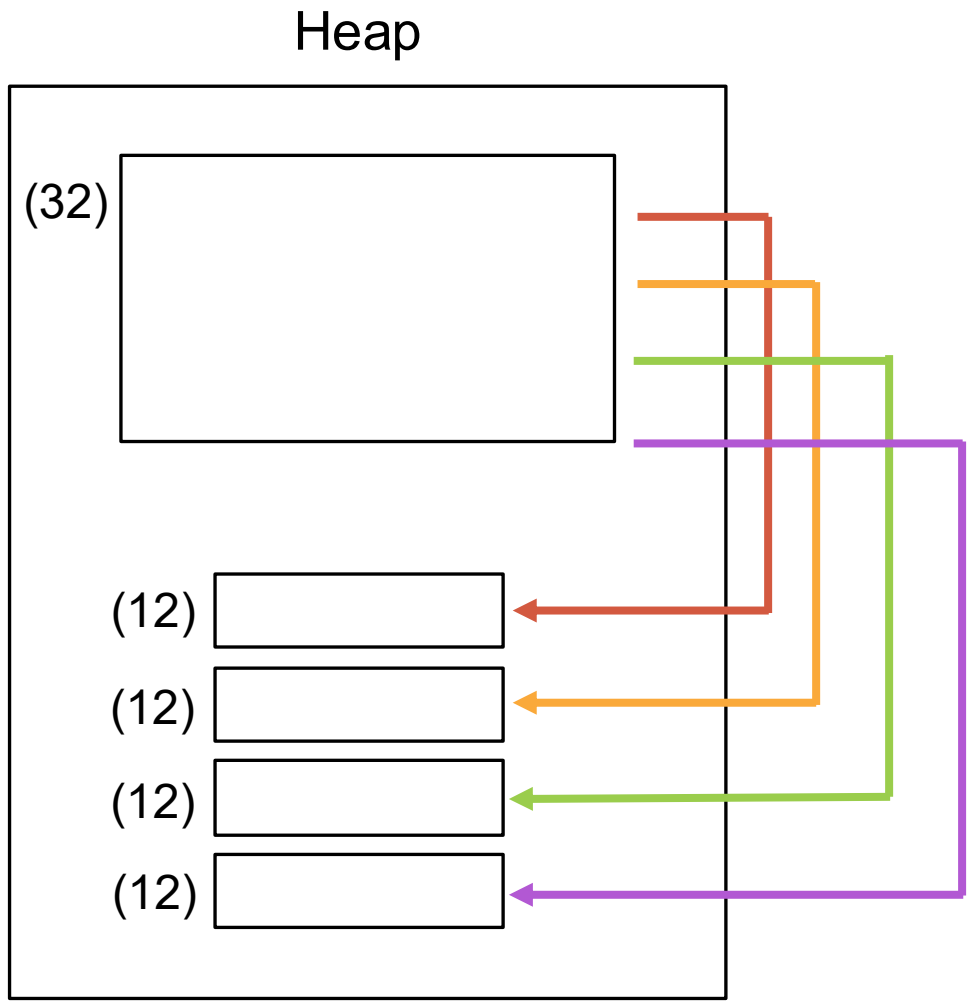
Heap

2D Arrays

Dynamically Allocated 2D Array (Method 2)

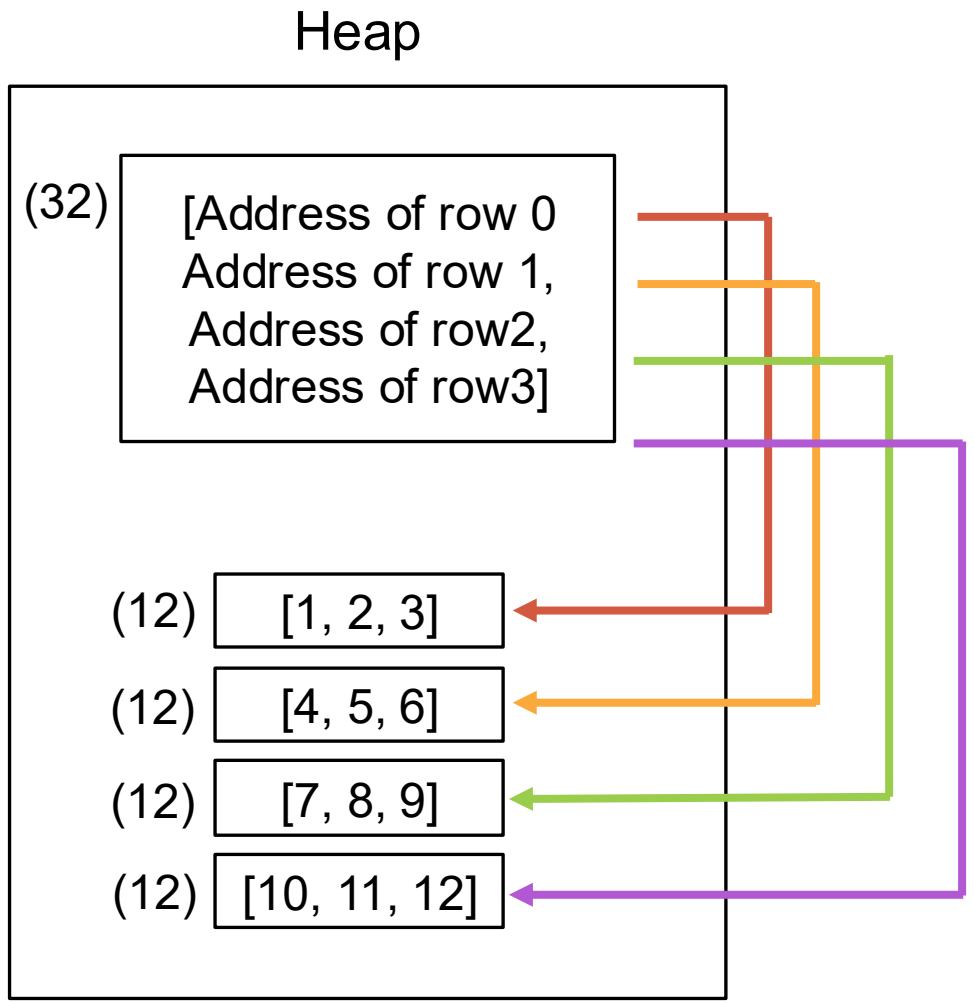
	Col 0	Col 1	Col 2
Row 0	1	2	3
Row 1	4	5	6
Row 2	7	8	9
Row 3	10	11	12

```
sizeof(int) = 4 bytes
```



Dynamically Allocated 2D Array (Method 2)

	Col 0	Col 1	Col 2
Row 0	1	2	3
Row 1	4	5	6
Row 2	7	8	9
Row 3	10	11	12



Dynamically Allocated 2D Array (Method 2)

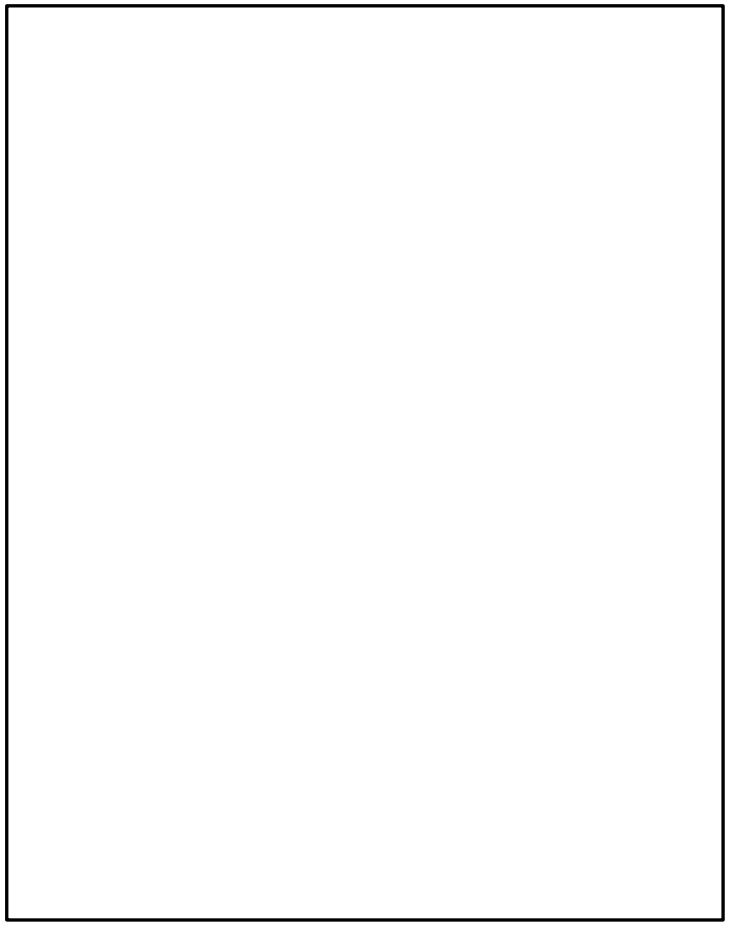
```c
#define NUM_ROWS 4
#define NUM_COLS 3

int main() {
    int32_t** two_d_array;
    int32_t i;

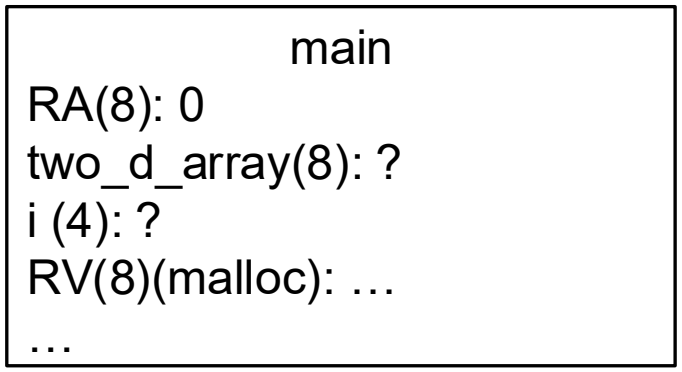
    two_d_array = malloc(sizeof(int32_t*) * NUM_ROWS);

    for (i = 0; i < NUM_ROWS; i++) {
        two_d_array[i] = malloc(sizeof(int32_t) * NUM_COLS);
}
```

Heap



Stack



(leaving out malloc frames for simplicity)

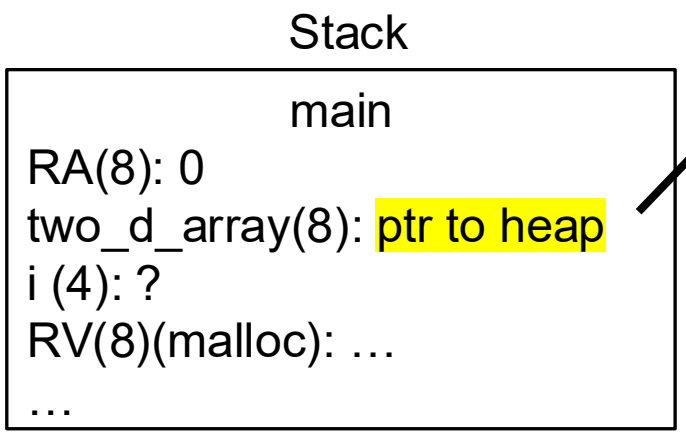
Dynamically Allocated 2D Array (Method 2)

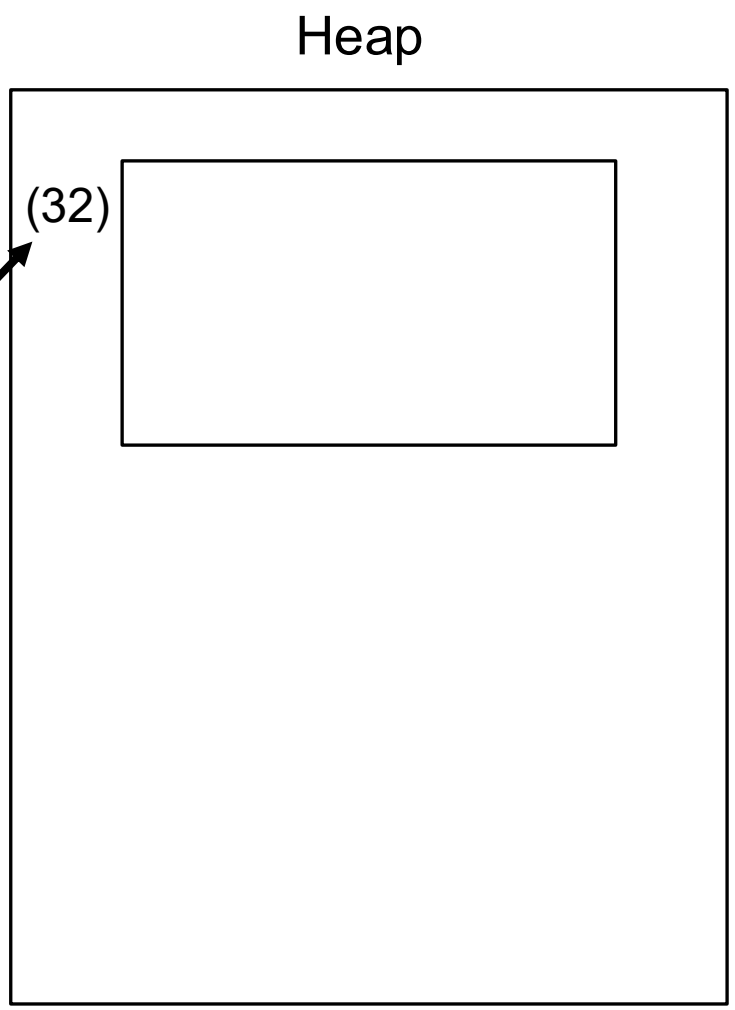
```c
#define NUM_ROWS 4
#define NUM_COLS 3

int main() {
    int32_t** two_d_array;
    int32_t i;

    two_d_array = malloc(sizeof(int32_t*) * NUM_ROWS);

    for (i = 0; i < NUM_ROWS; i++) {
        two_d_array[i] = malloc(sizeof(int32_t) * NUM_COLS);
}
```





(leaving out malloc frames for simplicity)

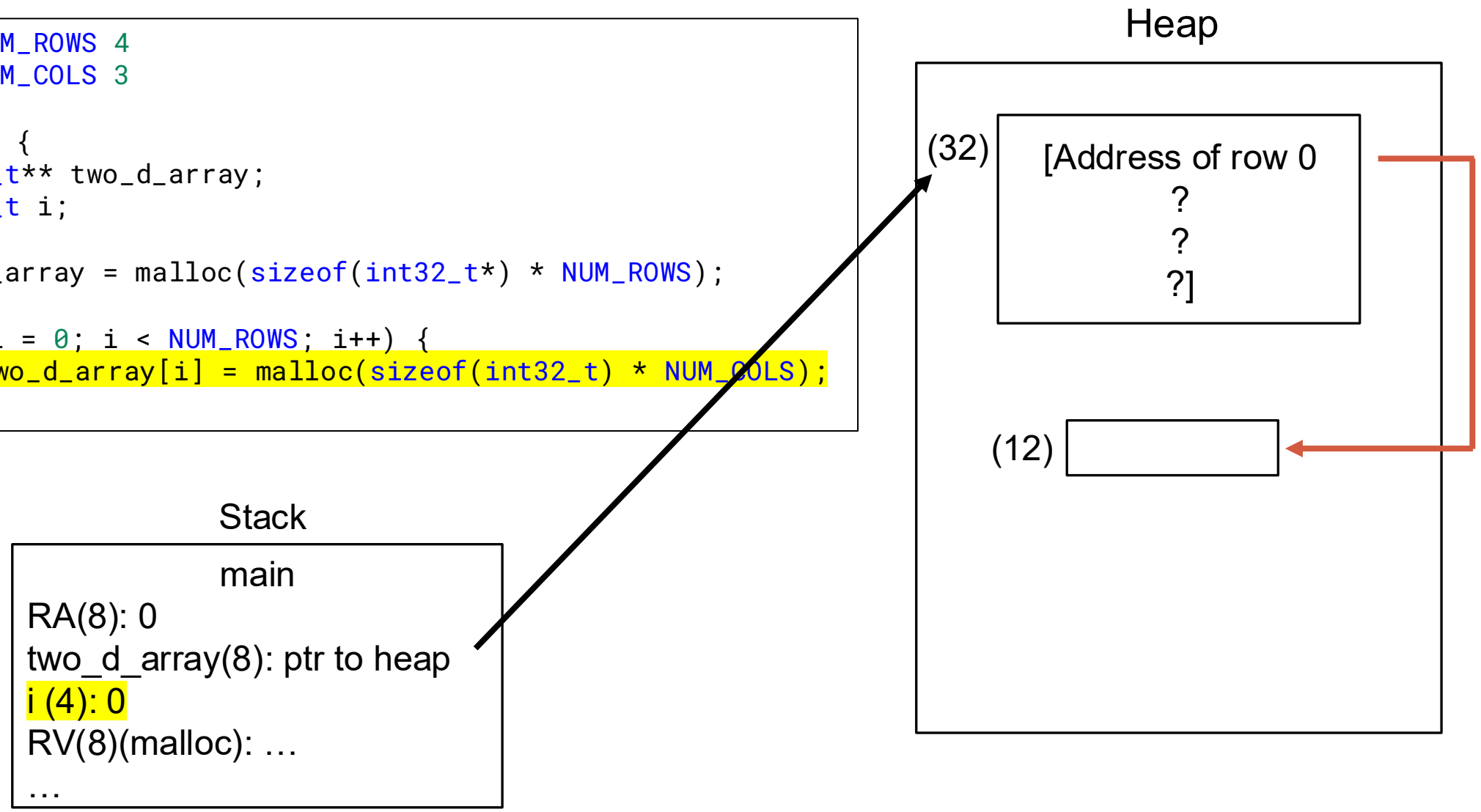
Dynamically Allocated 2D Array (Method 2)

```c
#define NUM_ROWS 4
#define NUM_COLS 3

int main() {
    int32_t** two_d_array;
    int32_t i;

    two_d_array = malloc(sizeof(int32_t*) * NUM_ROWS);

    for (i = 0; i < NUM_ROWS; i++) {
        two_d_array[i] = malloc(sizeof(int32_t) * NUM_COLS);
}
```



(leaving out malloc frames for simplicity)

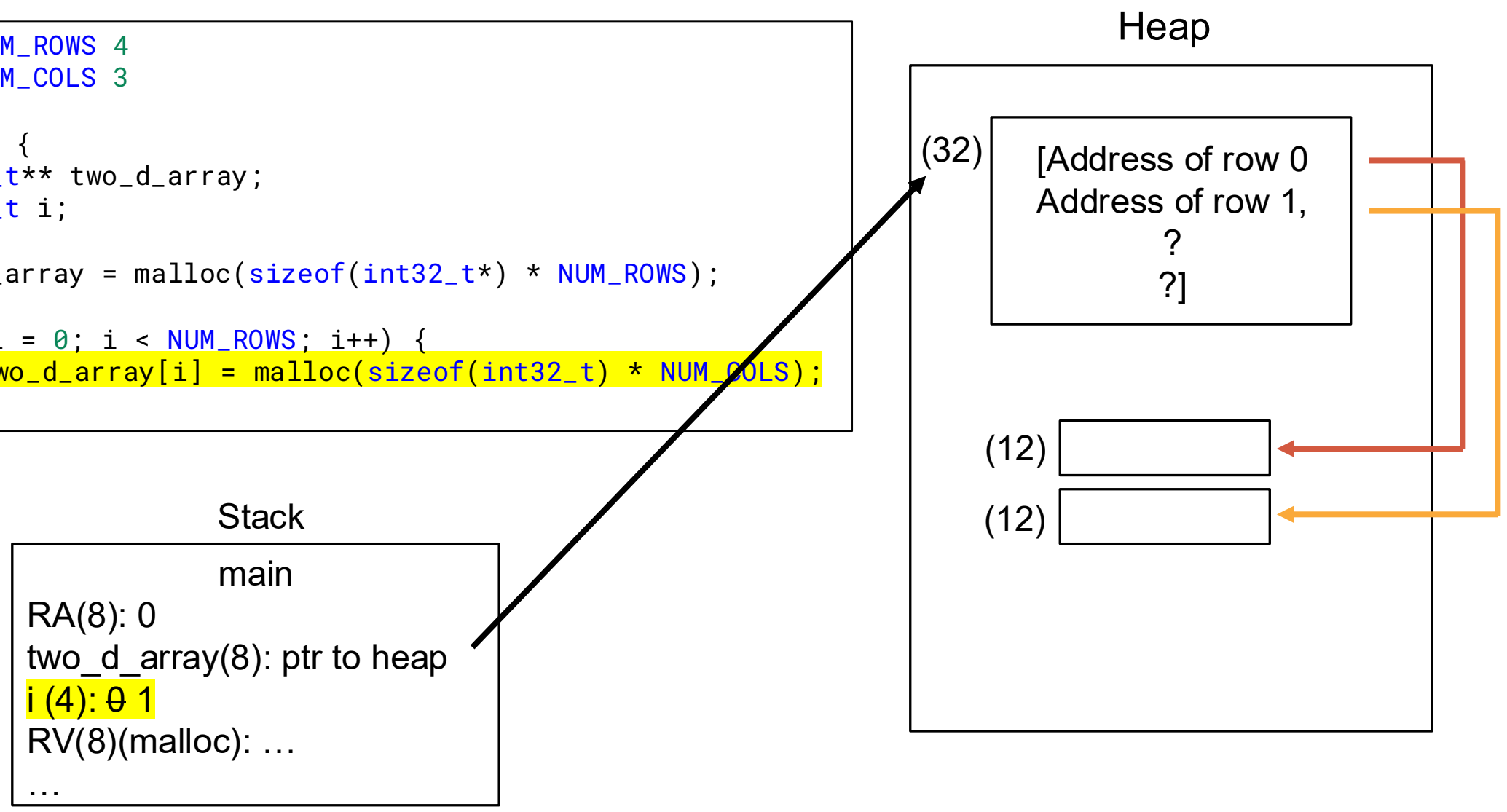
Dynamically Allocated 2D Array (Method 2)

```c
#define NUM_ROWS 4
#define NUM_COLS 3

int main() {
    int32_t** two_d_array;
    int32_t i;

    two_d_array = malloc(sizeof(int32_t*) * NUM_ROWS);

    for (i = 0; i < NUM_ROWS; i++) {
        two_d_array[i] = malloc(sizeof(int32_t) * NUM_COLS);
}
```



(leaving out malloc frames for simplicity)

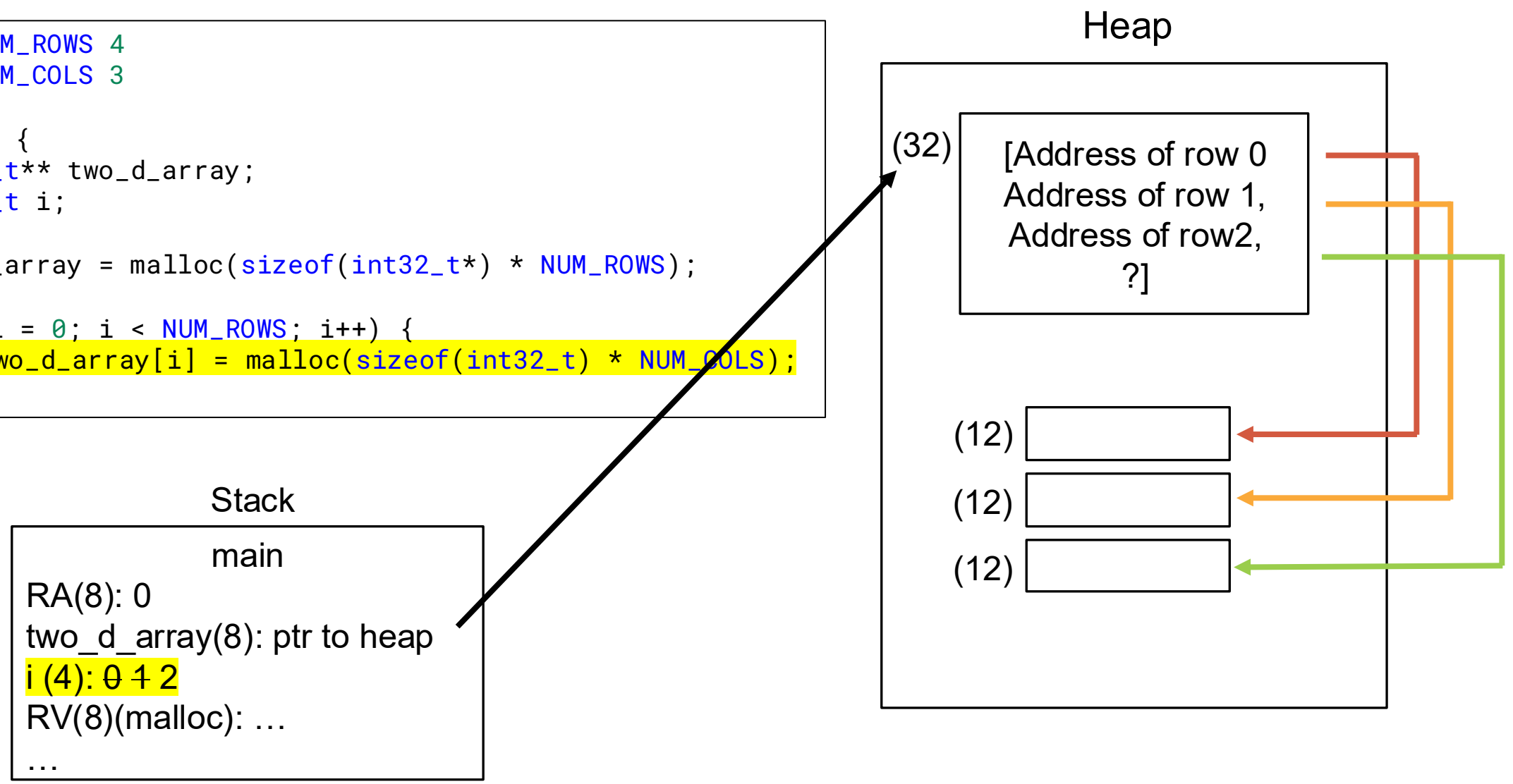
Dynamically Allocated 2D Array (Method 2)

```c
#define NUM_ROWS 4
#define NUM_COLS 3

int main() {
    int32_t** two_d_array;
    int32_t i;

    two_d_array = malloc(sizeof(int32_t*) * NUM_ROWS);

    for (i = 0; i < NUM_ROWS; i++) {
        two_d_array[i] = malloc(sizeof(int32_t) * NUM_COLS);
}
```



(leaving out malloc frames for simplicity)

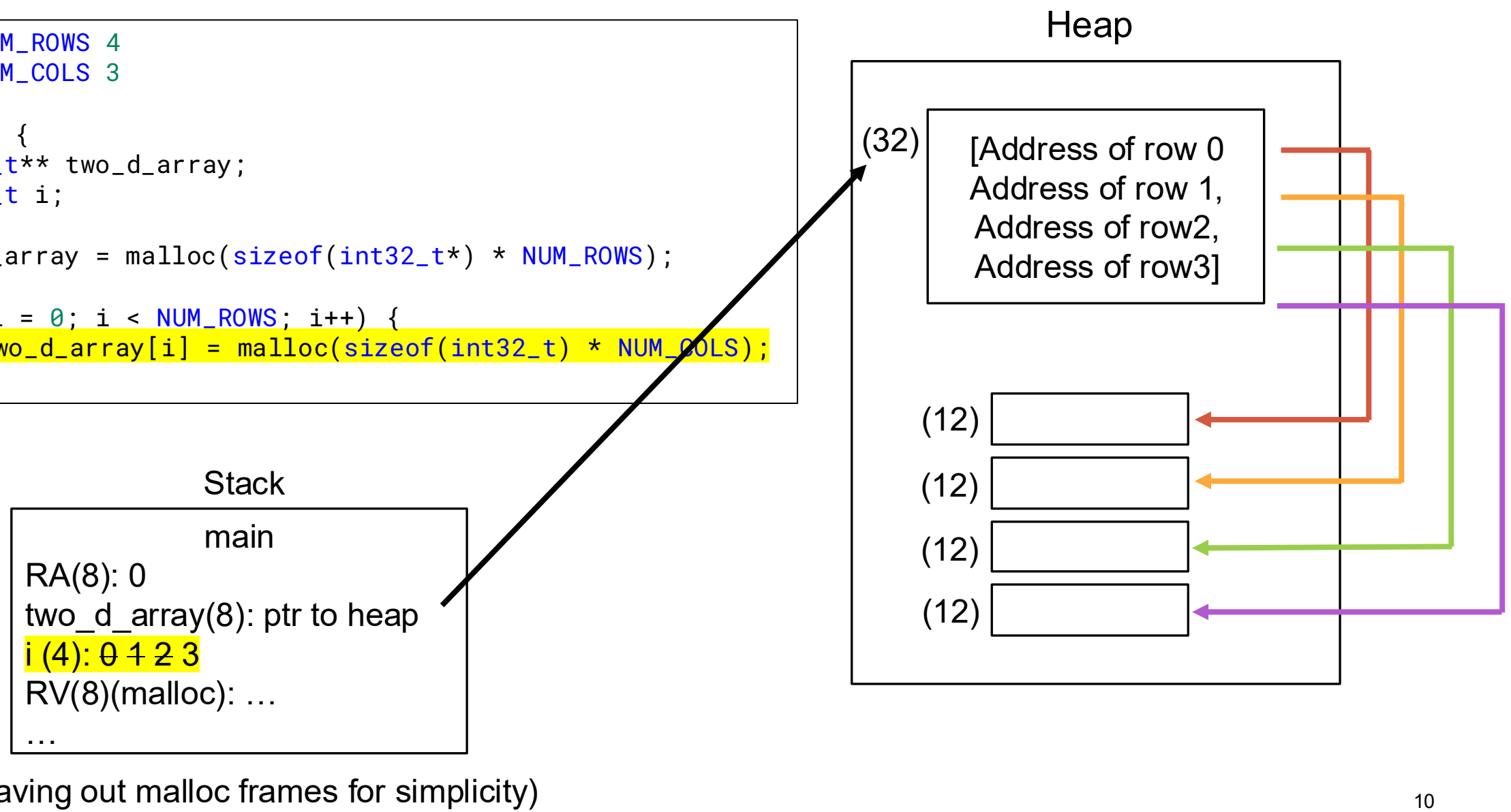
Dynamically Allocated 2D Array (Method 2)

```c
#define NUM_ROWS 4
#define NUM_COLS 3

int main() {
    int32_t** two_d_array;
    int32_t i;

    two_d_array = malloc(sizeof(int32_t*) * NUM_ROWS);

    for (i = 0; i < NUM_ROWS; i++) {
        two_d_array[i] = malloc(sizeof(int32_t) * NUM_COLS);
}
```



(leaving out malloc frames for simplicity)

```
cd lecture
git pull
```

OR

```
git clone https://github.com/COMP211-FA26/lecture.git
```

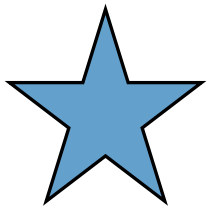
method2.c

- Implement free2d.
- Implement swap_rows.

Swap example

```
Before swap_rows(a, 5, 1, 4):
   0   1   2   3   4   5
  10  11  12  13  14  15
  20  21  22  23  24  25
  30  31  32  33  34  35
  40  41  42  43  44  45

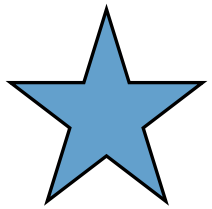
After swap:
   0   1   2   3   4   5
  40  41  42  43  44  45
  20  21  22  23  24  25
  30  31  32  33  34  35
  10  11  12  13  14  15
```



```c
static void free2d(int** a, int rows) {
   if (a == NULL) {
      return;
   }

   /* Free each separately allocated row */
   for (int i = 0; i < rows; i++) {
      free(a[i]);
   }

   /* Free the array of row pointers */
   free(a);
}
```



```c
void swap_rows(int** a, int rows, int r1, int r2) {
    if (a == NULL) {
        fprintf(stderr, "ERROR: array is NULL\n");
        exit(1);
    }

    if (r1 < 0 || r1 >= rows) {
        error_bad_row(r1, rows);
    }

    if (r2 < 0 || r2 >= rows) {
        error_bad_row(r2, rows);
    }

    int* temp = a[r1];
    a[r1] = a[r2];
    a[r2] = temp;
}
```

Heap Memory Management

Heap Memory Management

- The heap memory manager maintains a **free list** which contains the start address and size of contiguous unallocated blocks of memory in the heap.

- Initially, the heap consists of a single large free block, so the free list contains one entry representing the entire available heap.

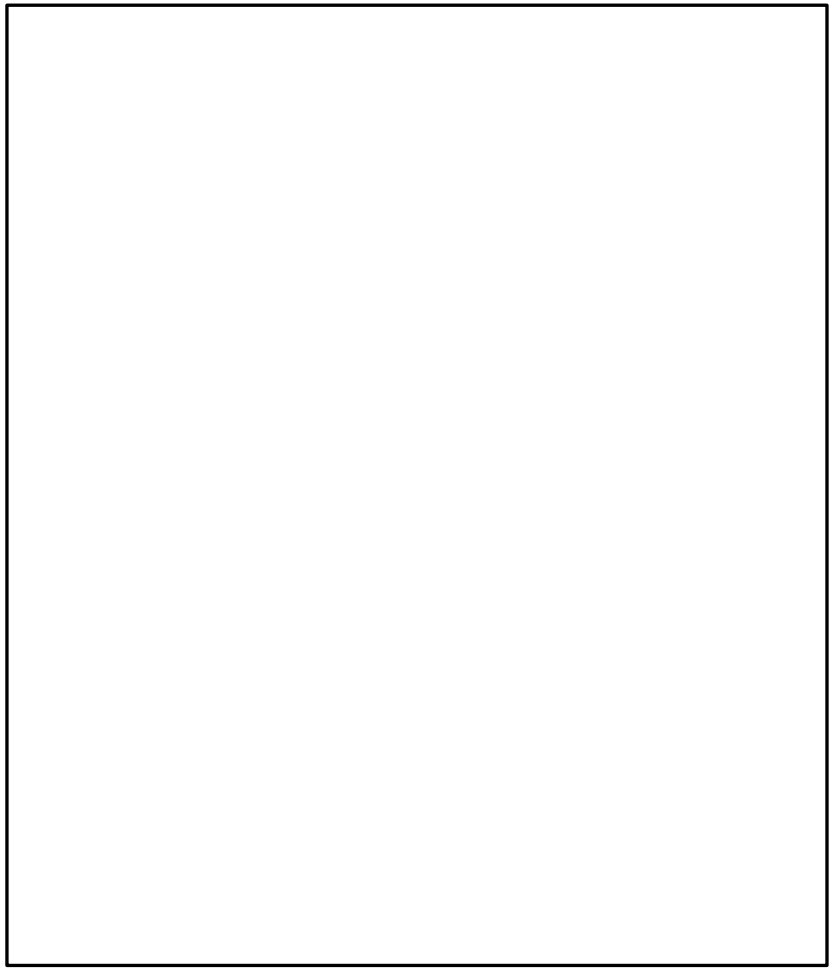
- After performing a series of malloc and free operations, the heap can become **fragmented** (free spaces end up scattered among allocated blocks).

Heap Memory Management Example

- Initial State
  - Heap size: 1000 bytes (this is an oversimplification)
  - Free List: One block covering the entire heap

Initial State

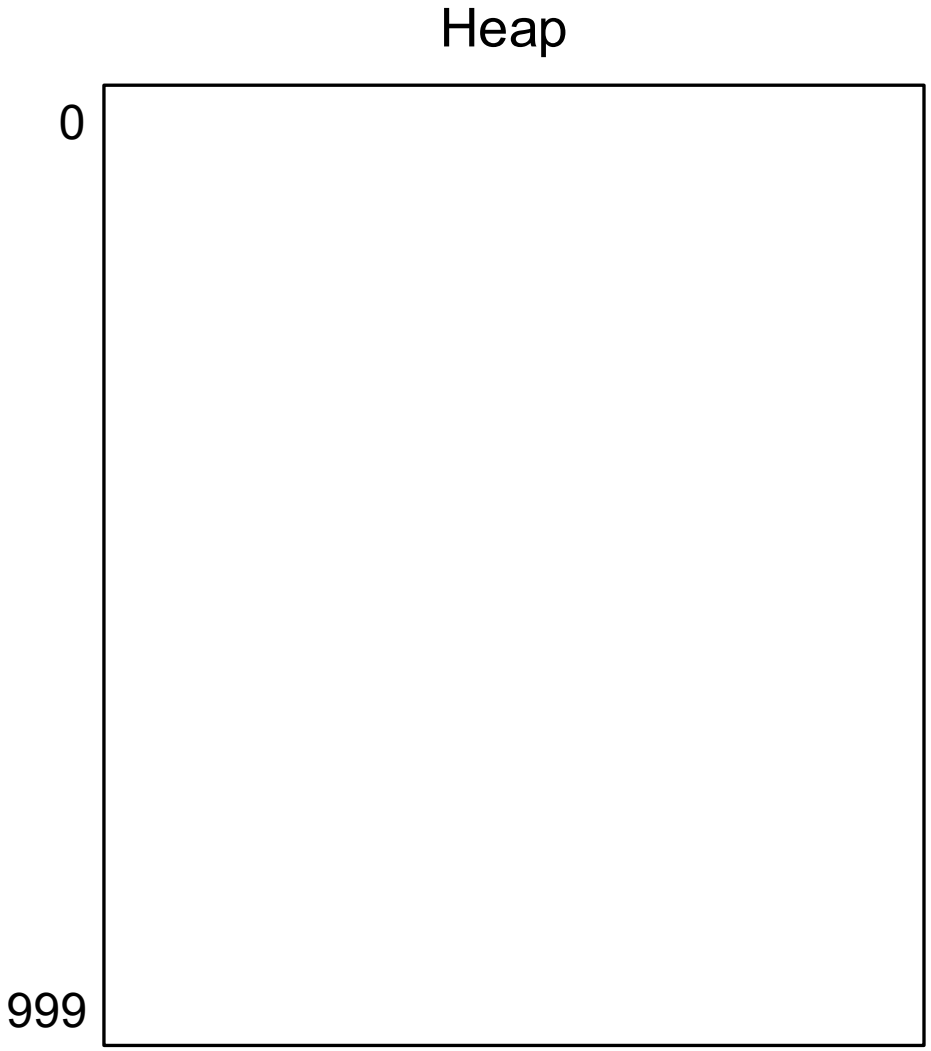
Heap



Free list

Bytes 0 - 999

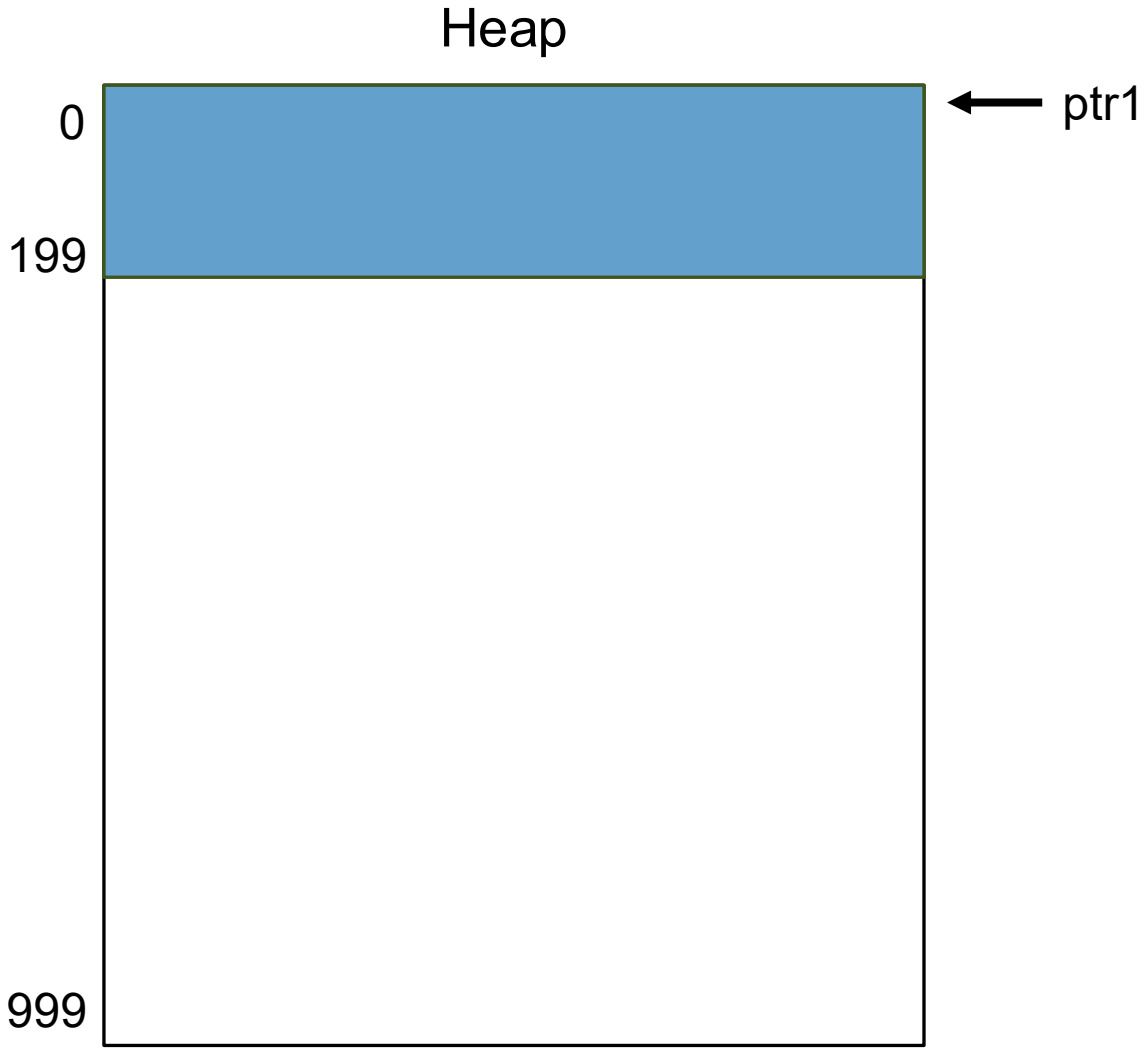
Operation 1: ptr1 = malloc(200)



Free list

Bytes 0 - 999

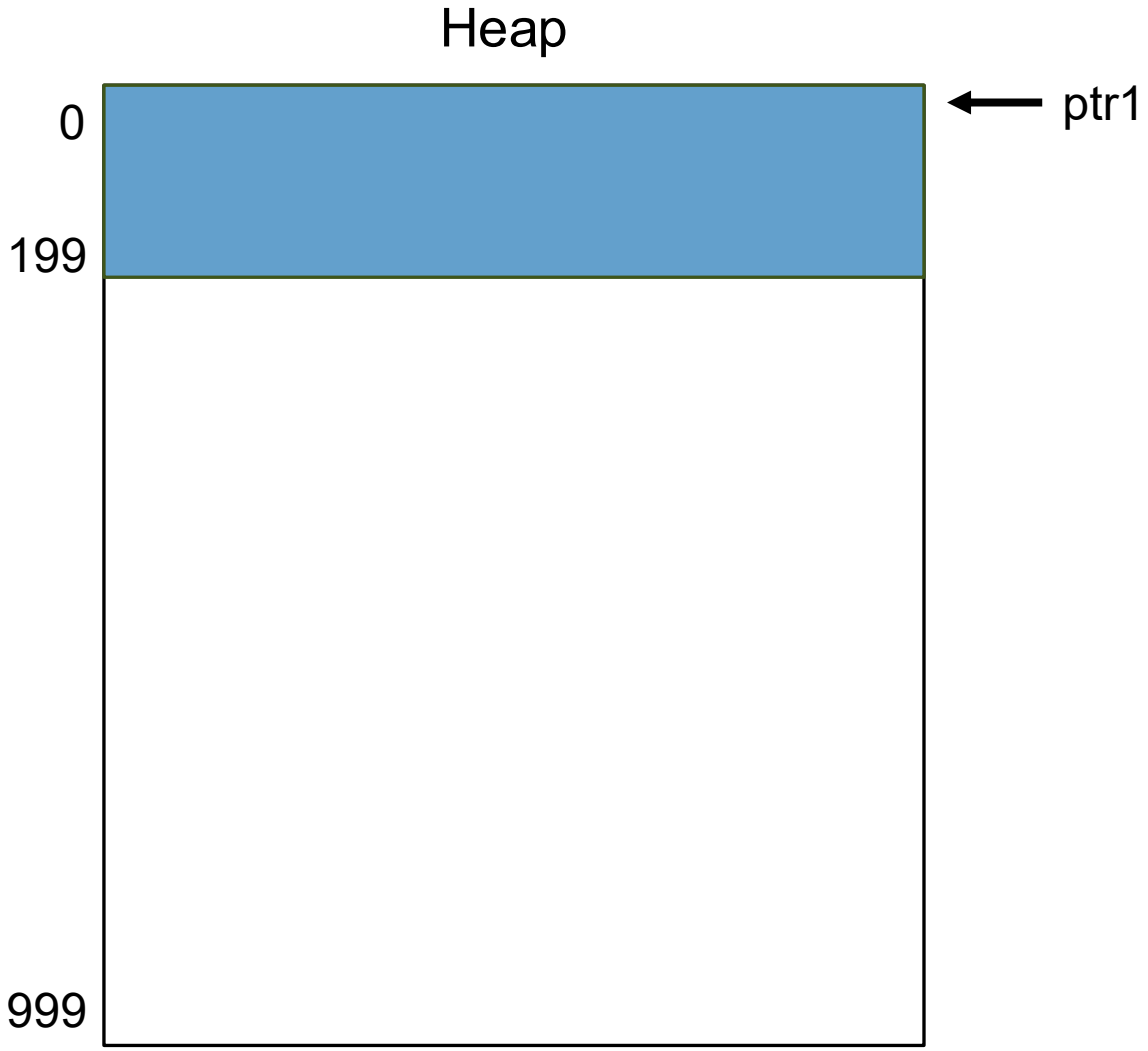
Operation 1: ptr1 = malloc(200)



Free list

Bytes 200 - 999

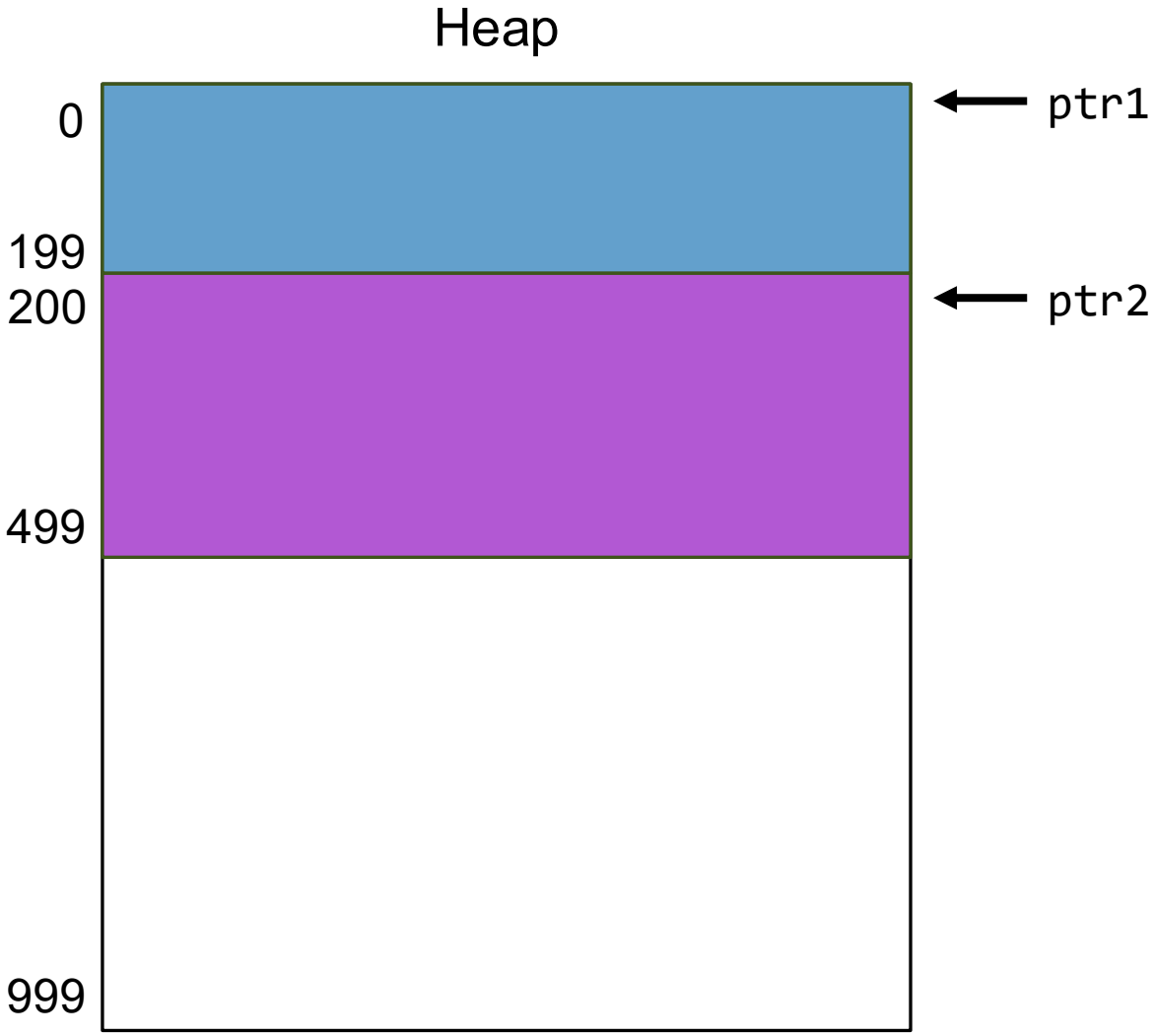
Operation 2: ptr2 = malloc(300)



Free list

Bytes 200 - 999

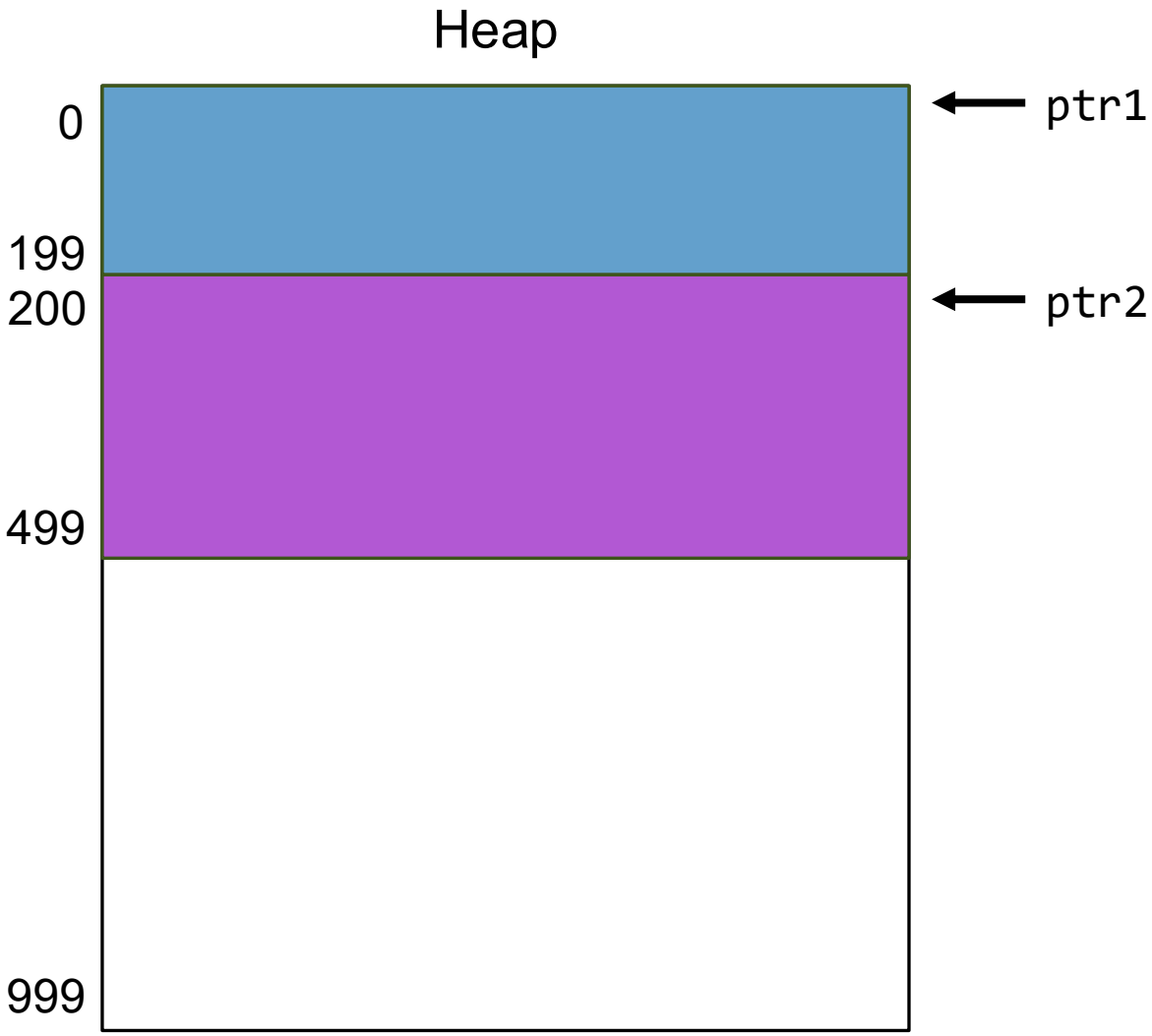
Operation 2: ptr2 = malloc(300)



Free list

Bytes 500 - 999

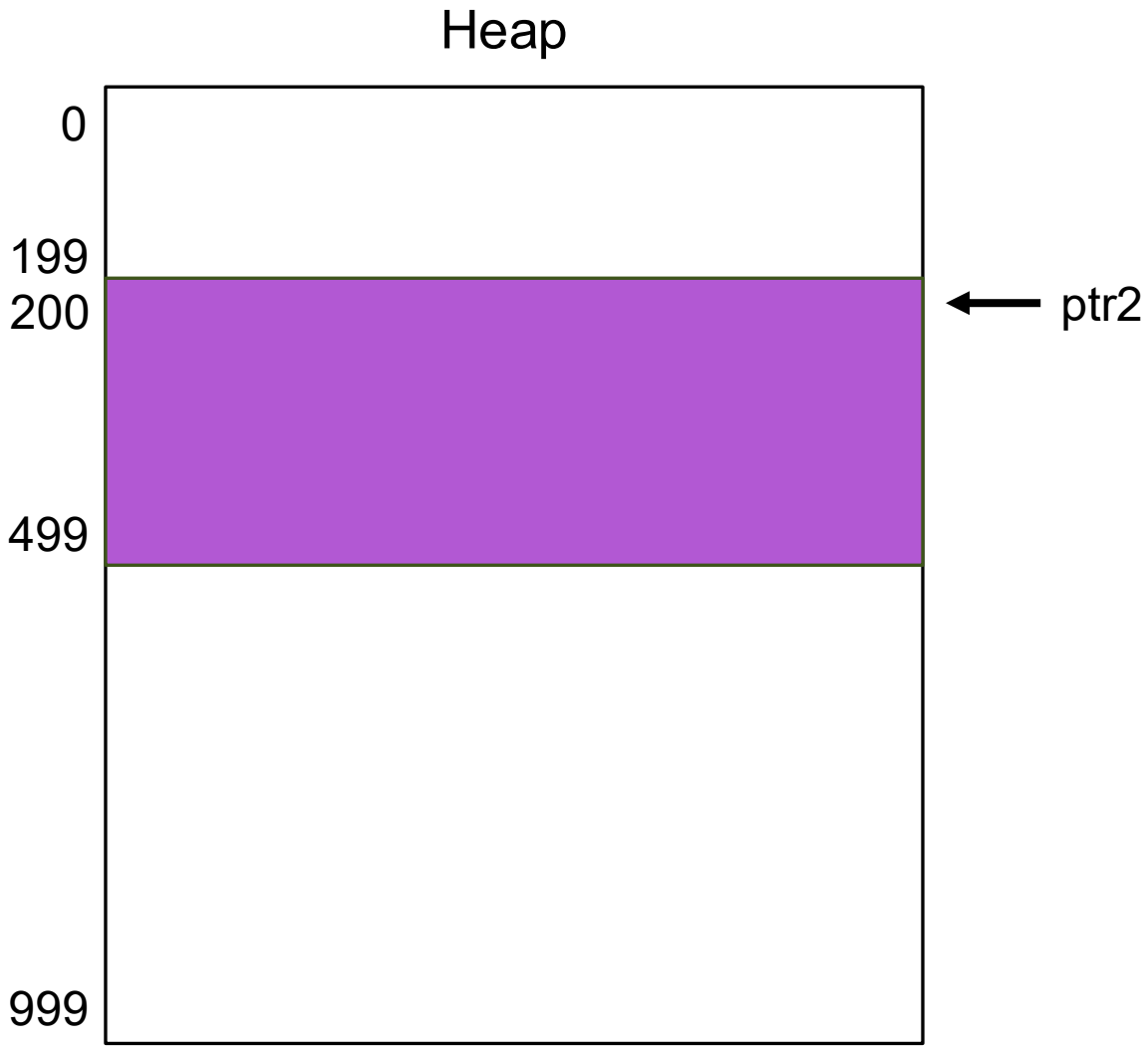
Operation 3: free(ptr1)



Free list

Bytes 500 - 999

Operation 3: free(ptr1)

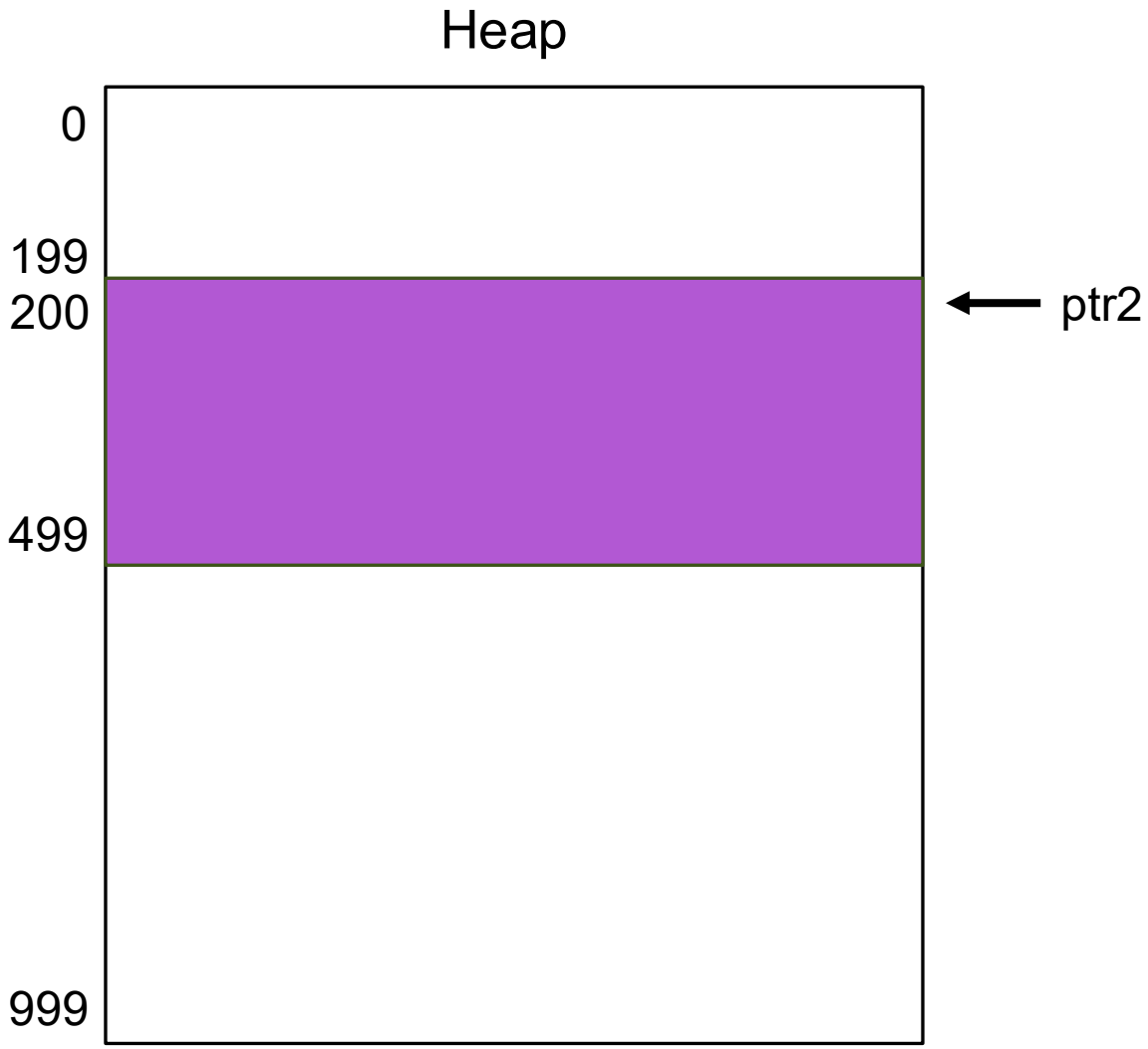


Free list

Bytes 0 - 199

Bytes 500 - 999

Operation 4: ptr3 = malloc(100)

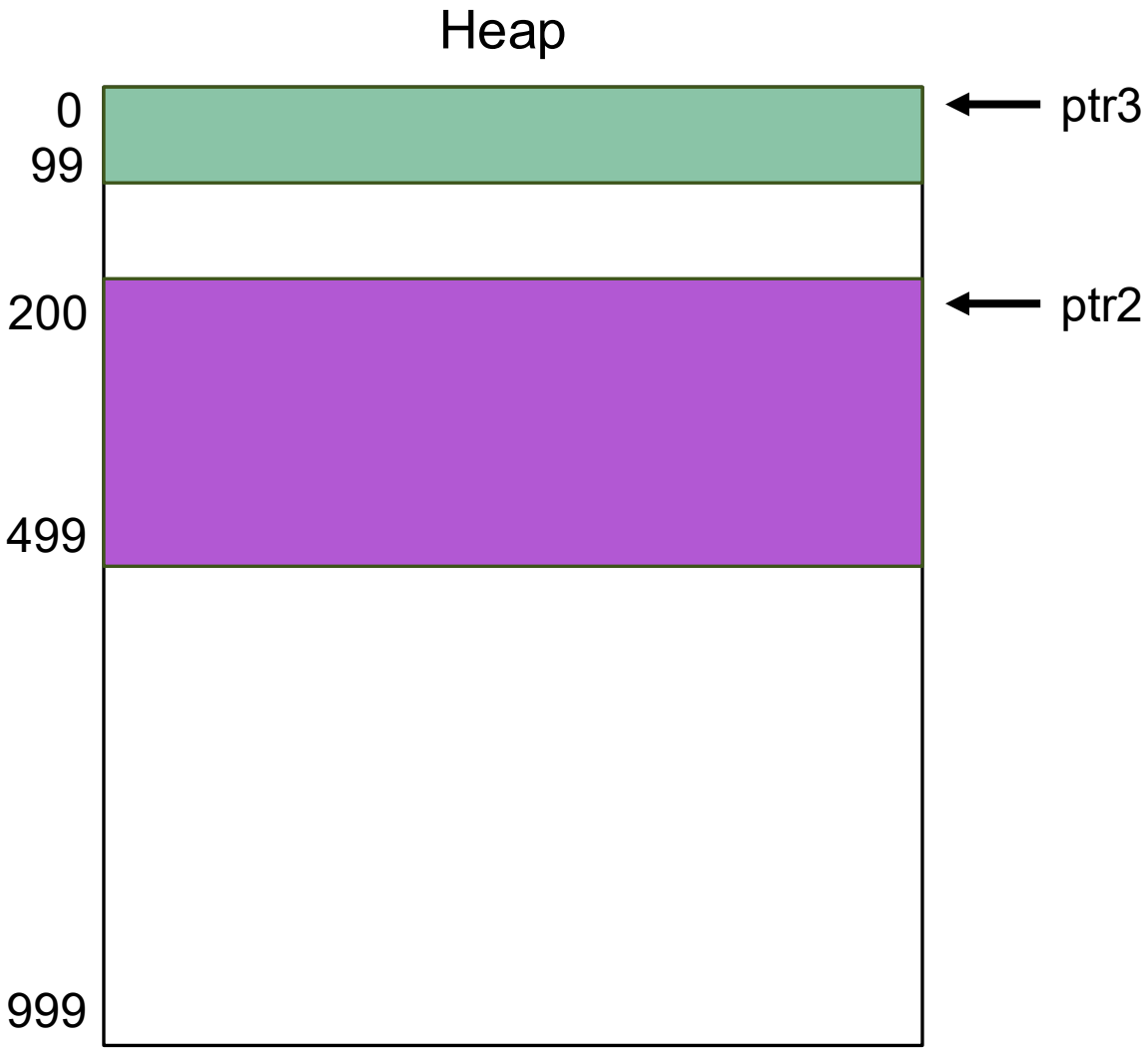


Free list

Bytes 0 - 199

Bytes 500 - 999

Operation 4: ptr3 = malloc(100)

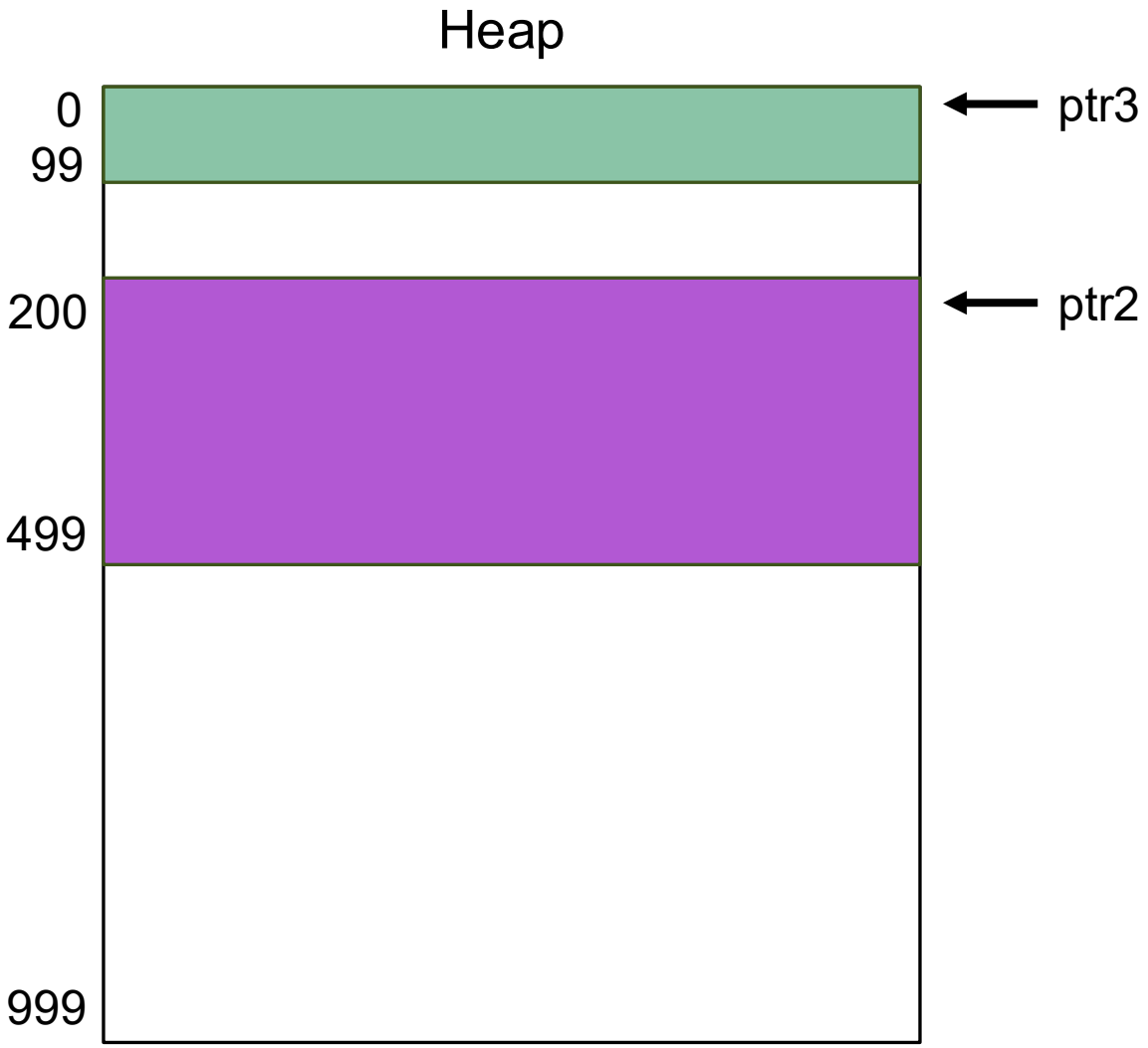


Free list

Bytes 100 - 199

Bytes 500 - 999

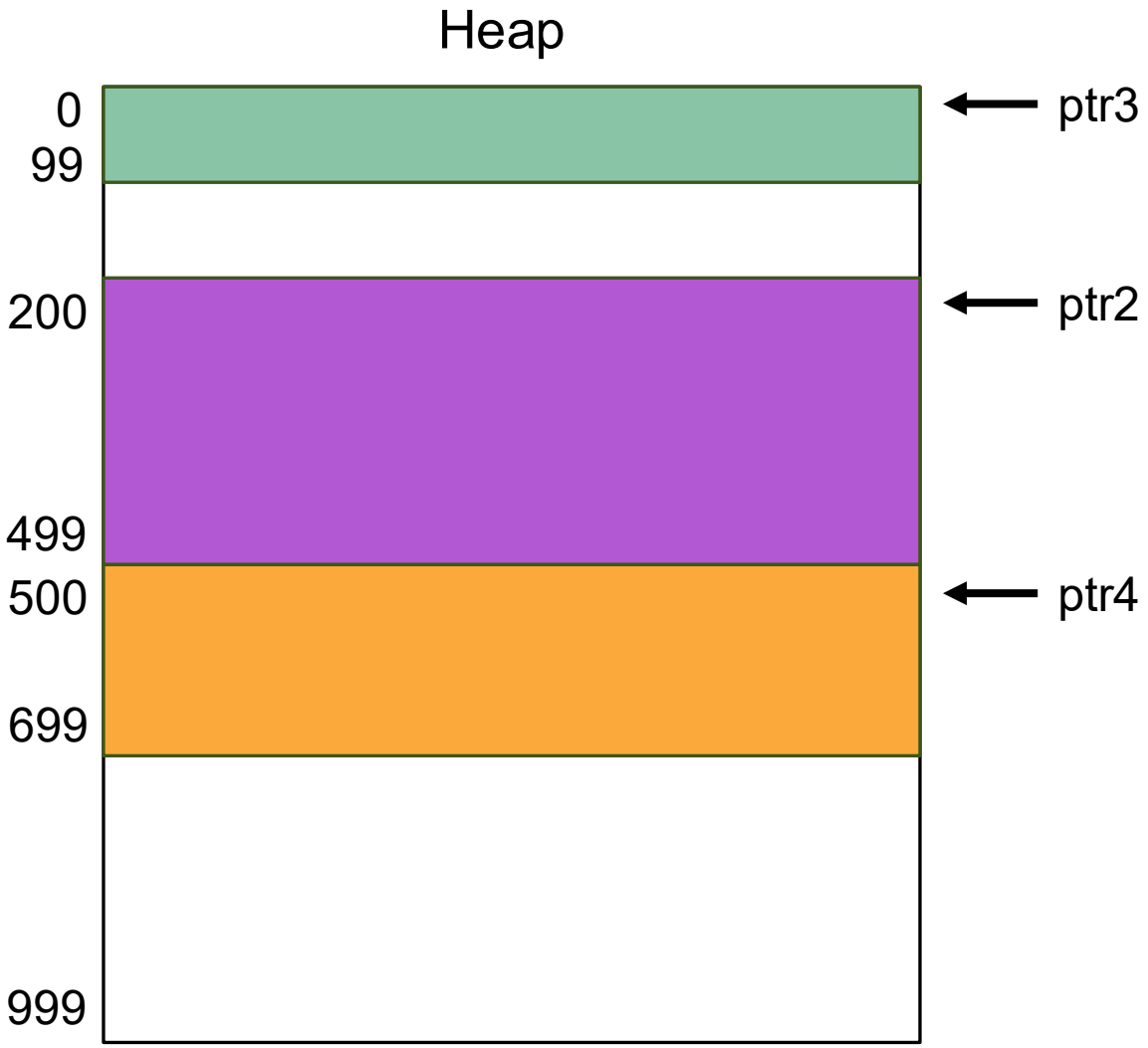
Operation 5: ptr4 = malloc(200)



Free list

- Bytes 100 - 199
- Bytes 500 - 999

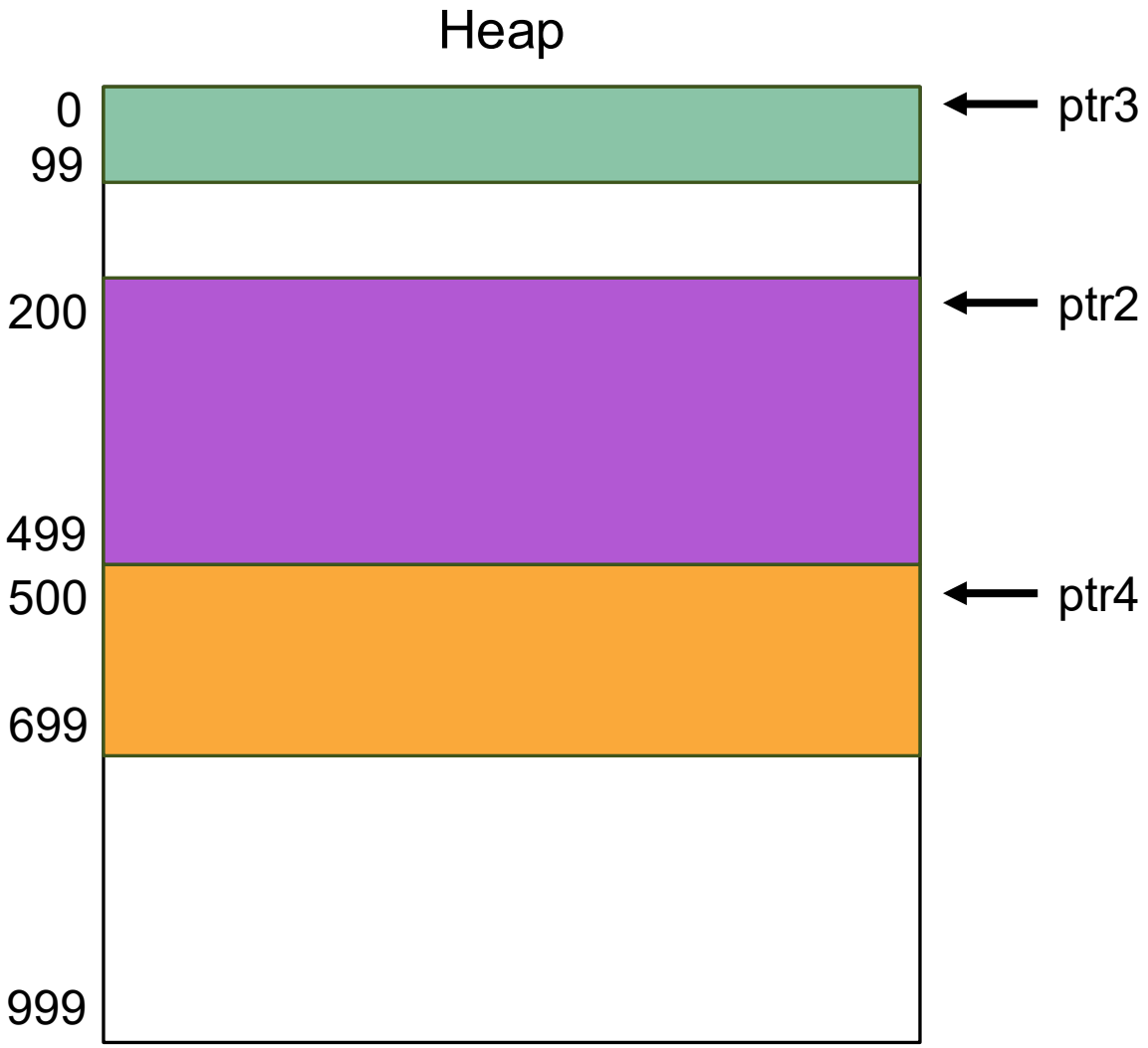
Operation 5: ptr4 = malloc(200)



Free list

- Bytes 100 - 199
- Bytes 700 - 999

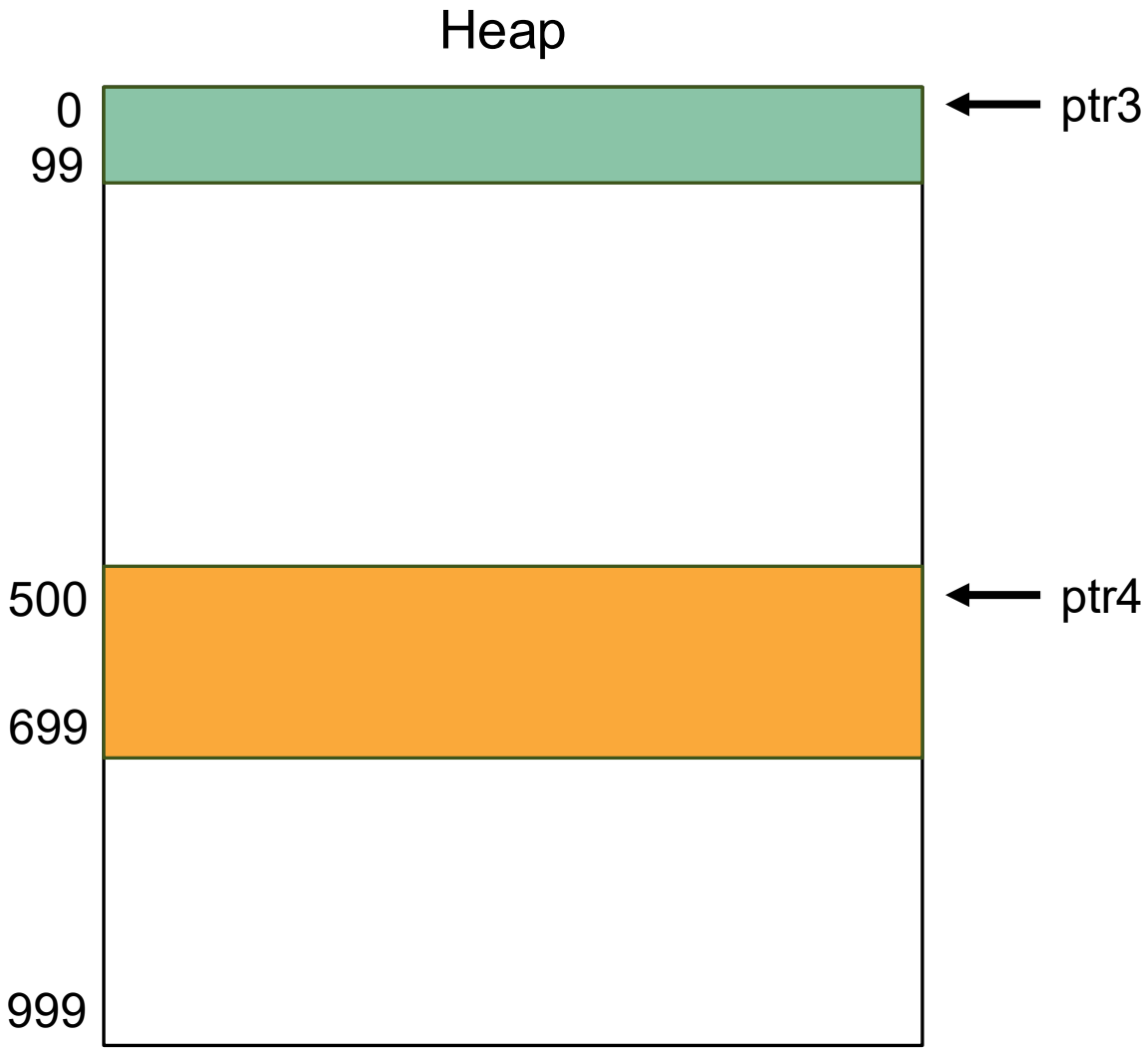
Operation 6: free(ptr2)



Free list

- Bytes 100 - 199
- Bytes 700 - 999

Operation 6: free(ptr2)



Free list

- Bytes 100 - 499
- Bytes 700 - 999

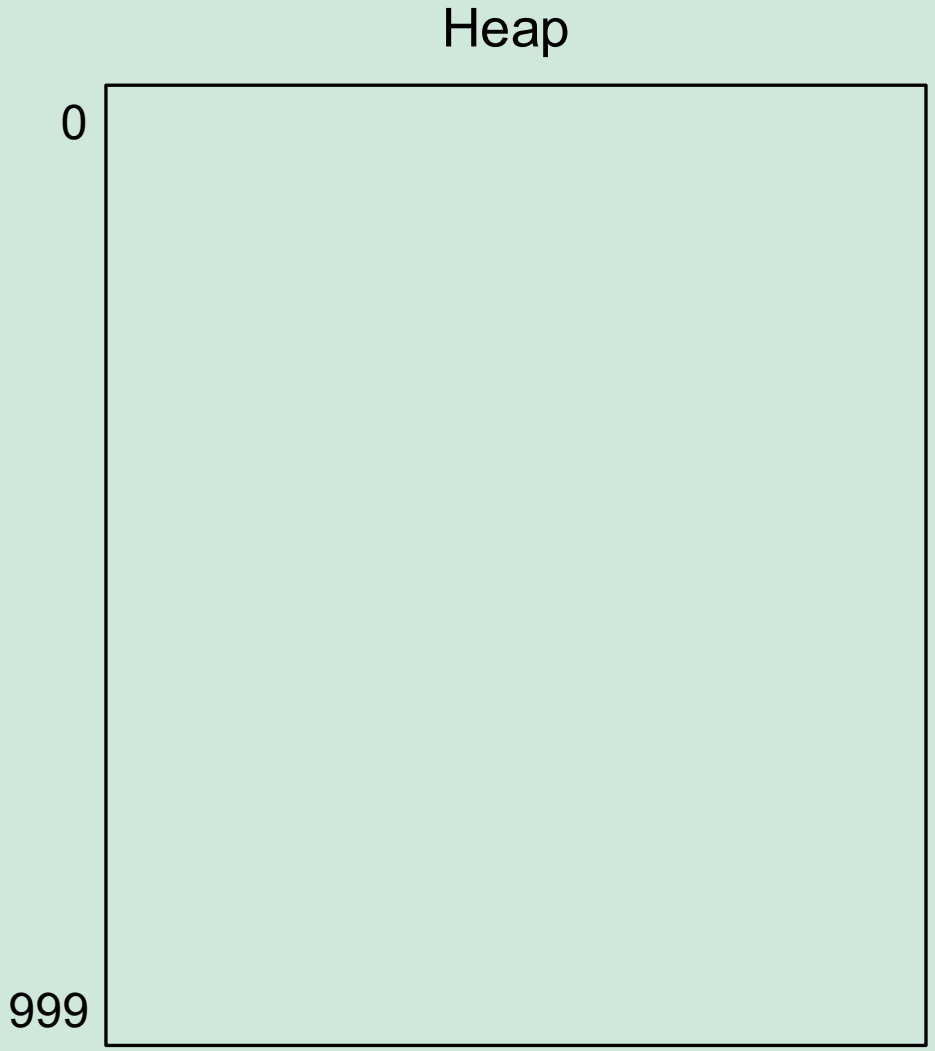
Coalescing Free Blocks

Two adjacent blocks are now free (100 – 199 and 200 – 499), so they can be merged into one block.

First Fit Allocation Strategy

- This example used the first fit allocation strategy, in which the allocator
  - Starts at the beginning of the free list and checks each free block one by one.
  - Selects the first block that is large enough to accommodate the request.
  - If the chosen free block is larger than needed, it may be split into two parts:
    - one part is allocated
    - and the remaining part is left in the free list as a smaller free block.

Initial State



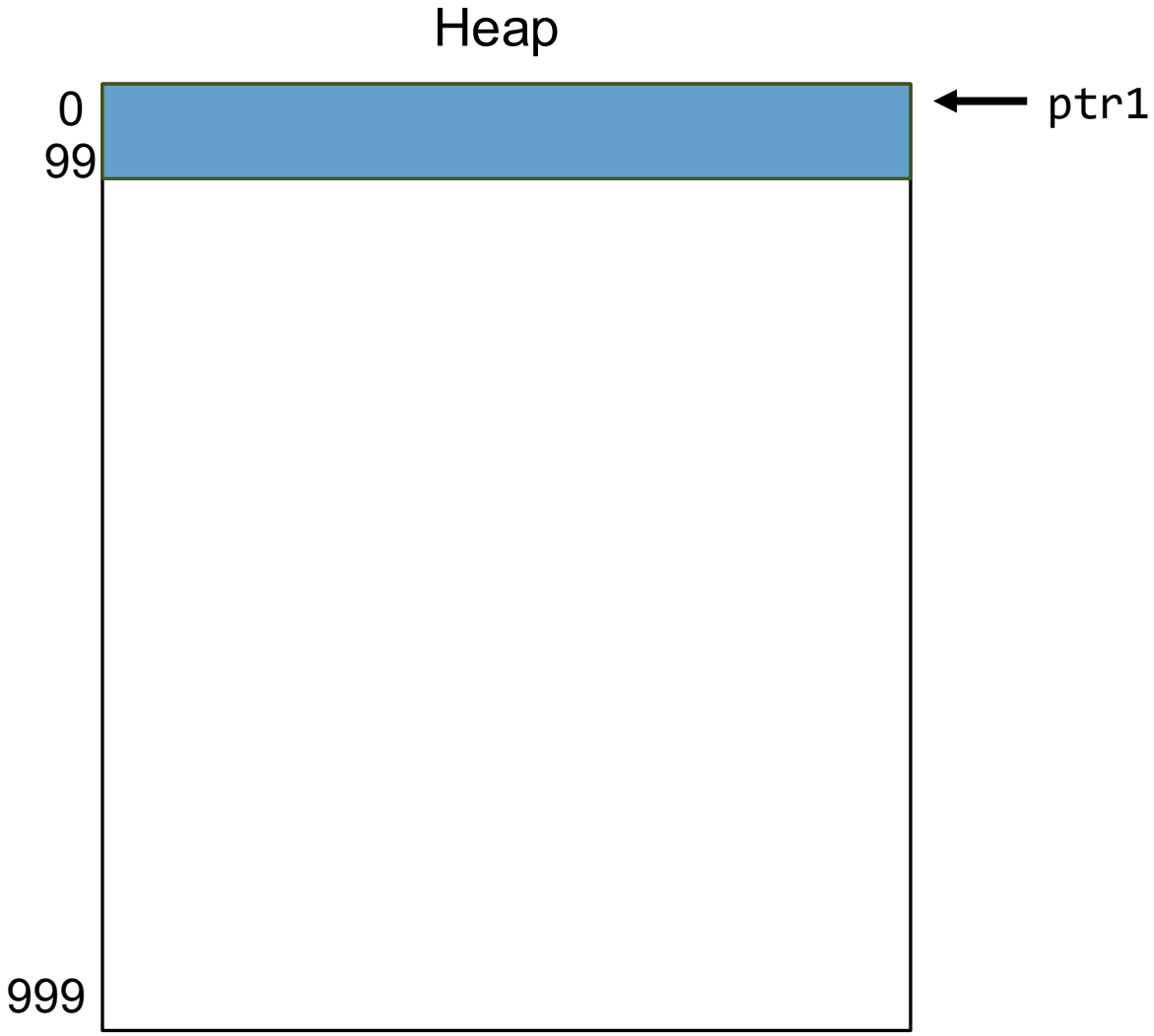
Free list

Bytes 0 - 999

Draw the state of the heap and the free list after the following operations are executed.

```
ptr1 = malloc(100);
free(ptr1);
ptr2 = malloc(200);
ptr3 = malloc(400);
free(ptr2);
ptr4 = malloc(300);
ptr5 = malloc(50);
```

Operation 1: `ptr1 = malloc(100);`



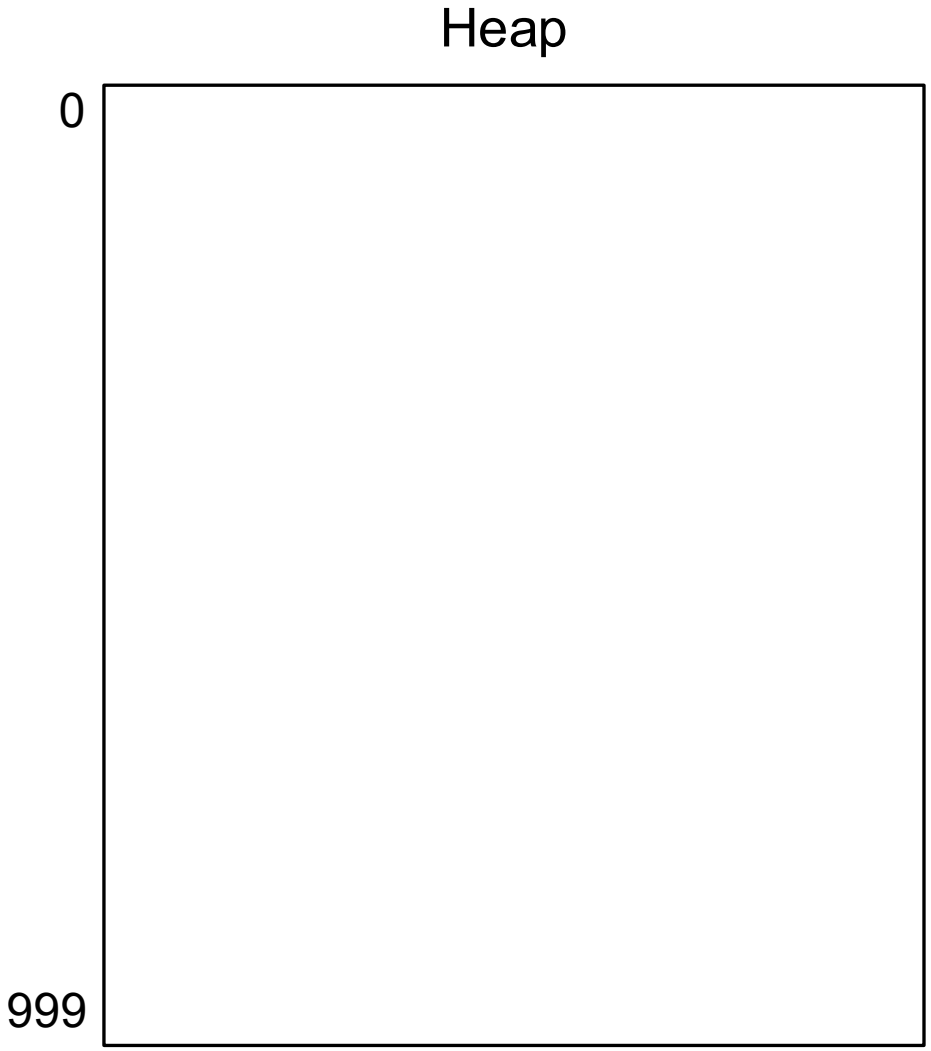
Free list

Bytes 100 - 999

Draw the state of the heap and the free list after the following operations are executed.

```
ptr1 = malloc(100);
free(ptr1);
ptr2 = malloc(200);
ptr3 = malloc(400);
free(ptr2);
ptr4 = malloc(300);
ptr5 = malloc(50);
```

Operation 2: `free(ptr1);`



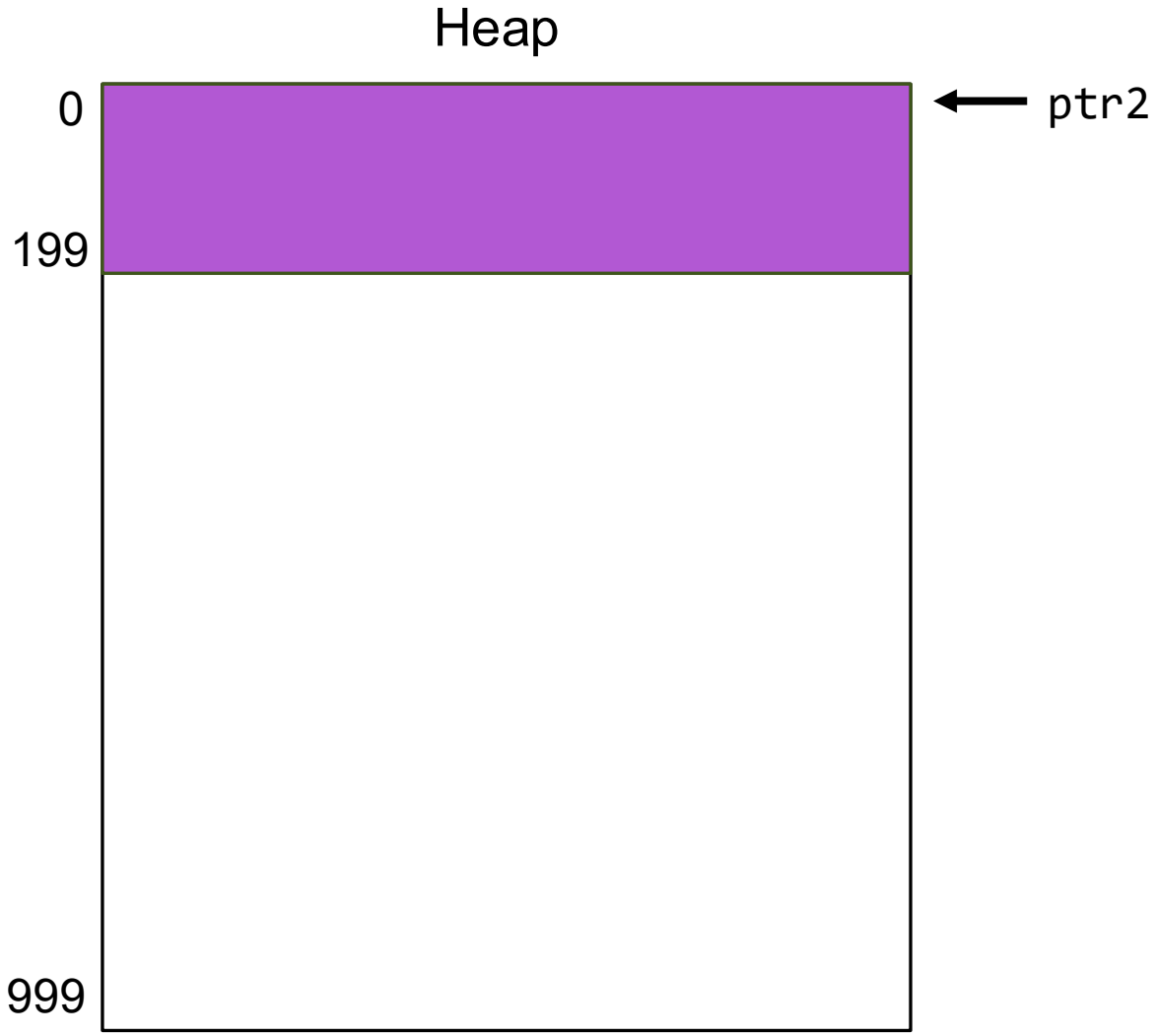
Free list

Bytes 0 - 999

Draw the state of the heap and the free list after the following operations are executed.

```
ptr1 = malloc(100);
free(ptr1);
ptr2 = malloc(200);
ptr3 = malloc(400);
free(ptr2);
ptr4 = malloc(300);
ptr5 = malloc(50);
```

Operation 3: `ptr2 = malloc(200);`



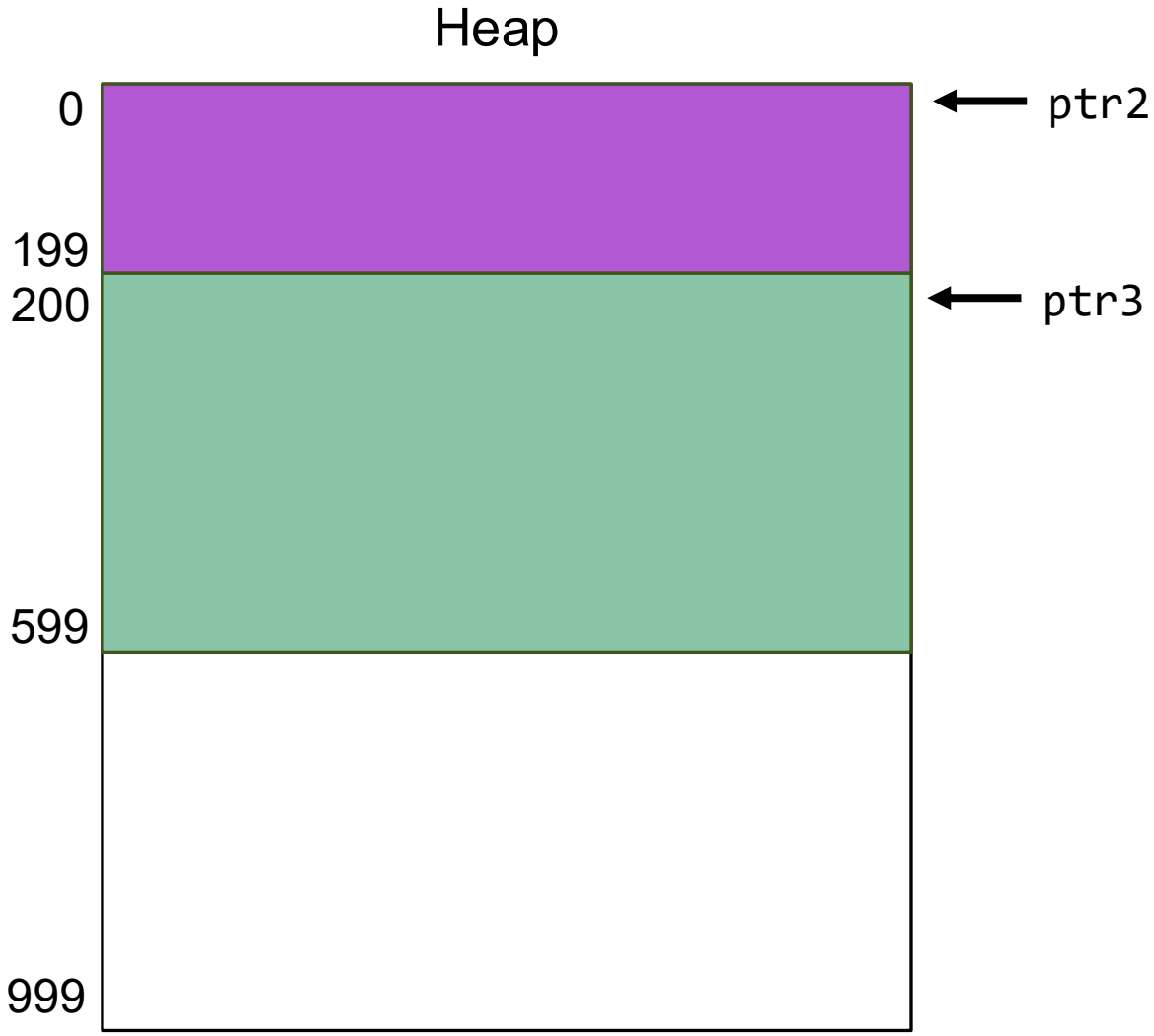
Free list

Bytes 200 - 999

Draw the state of the heap and the free list after the following operations are executed.

```
ptr1 = malloc(100);
free(ptr1);
ptr2 = malloc(200);
ptr3 = malloc(400);
free(ptr2);
ptr4 = malloc(300);
ptr5 = malloc(50);
```

Operation 4: `ptr3 = malloc(400);`



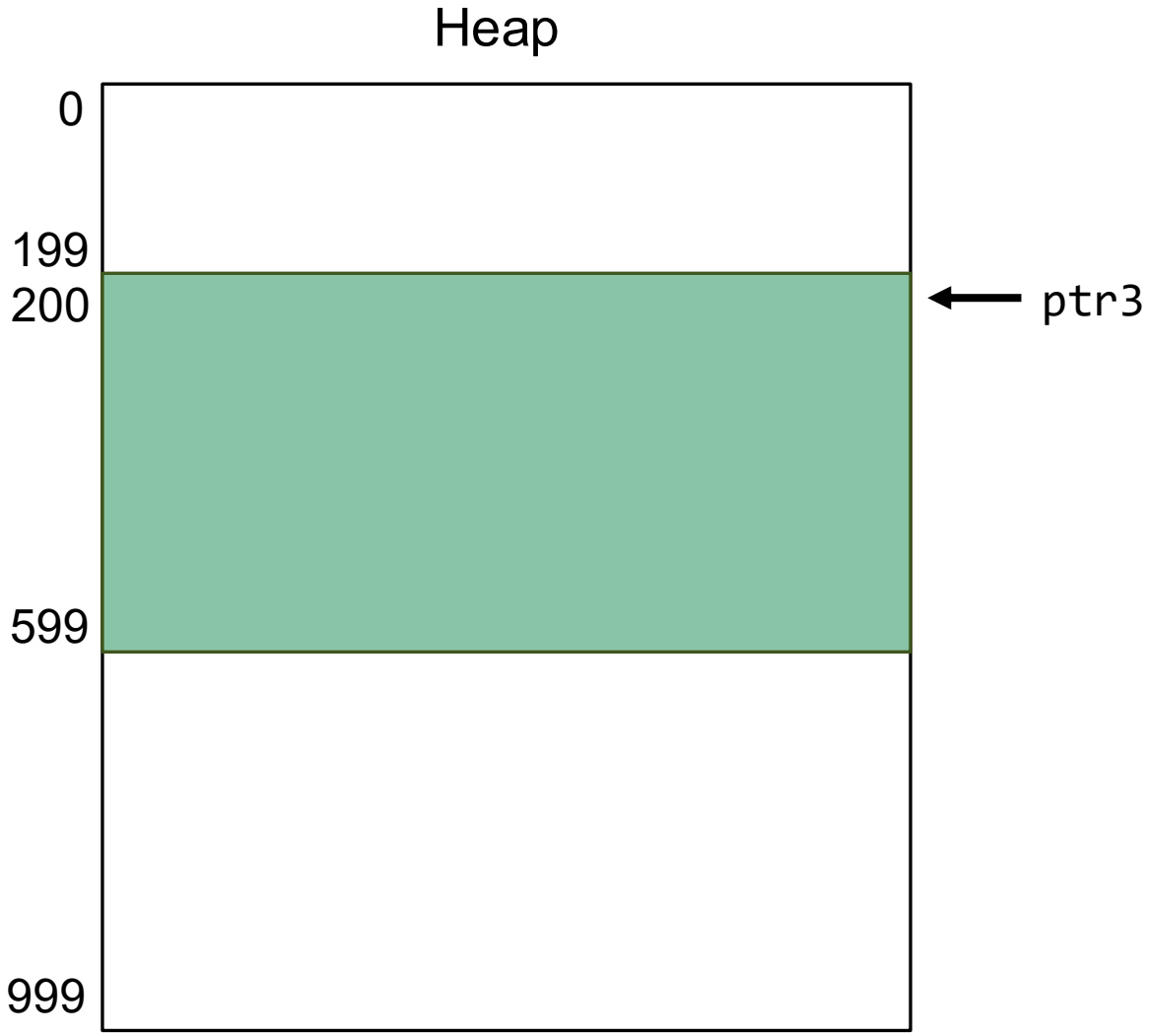
Free list

Bytes 600 - 999

Draw the state of the heap and the free list after the following operations are executed.

```
ptr1 = malloc(100);
free(ptr1);
ptr2 = malloc(200);
ptr3 = malloc(400);
free(ptr2);
ptr4 = malloc(300);
ptr5 = malloc(50);
```

Operation 5: `free(ptr2);`



Free list

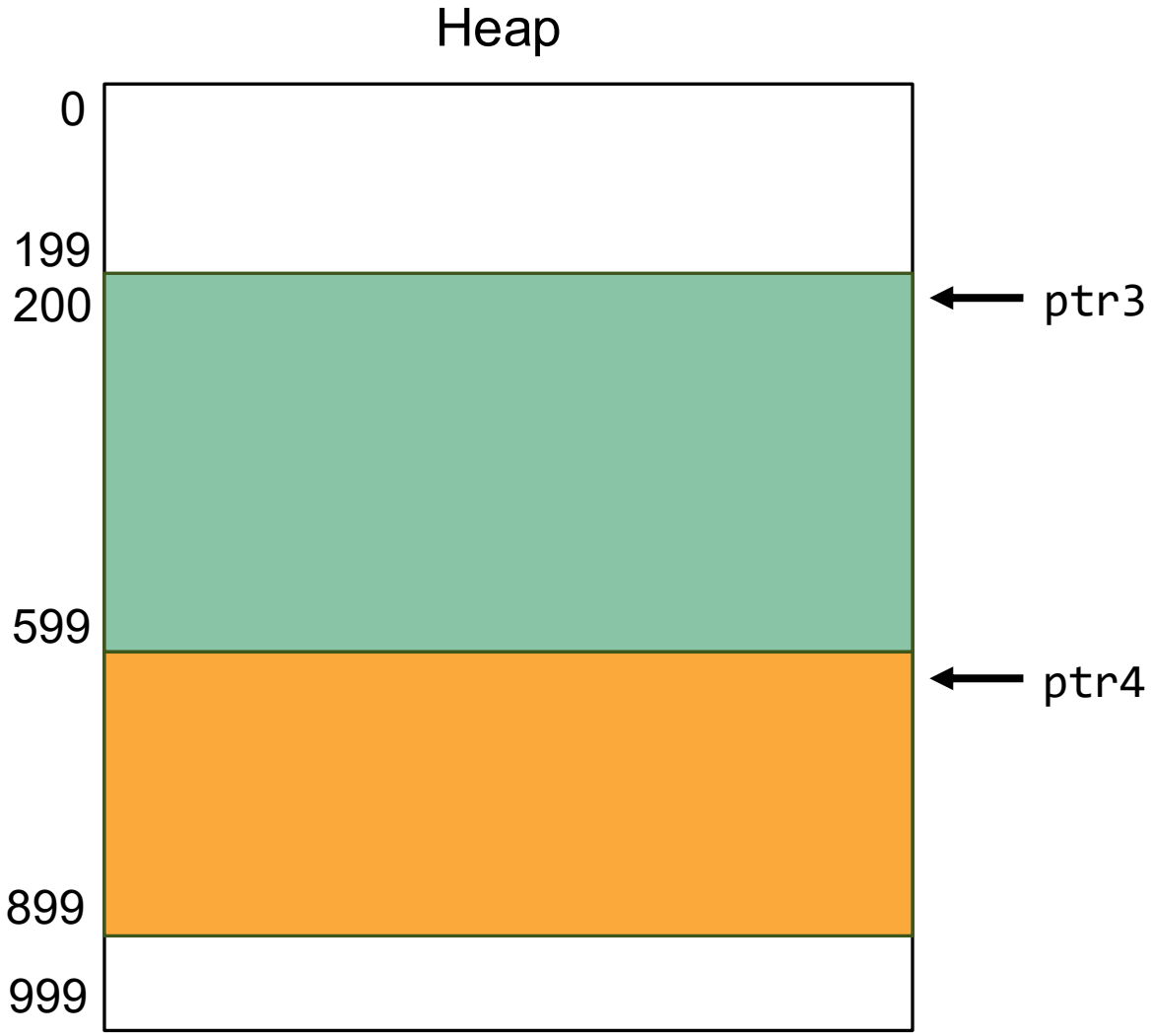
Bytes 0 - 199

Bytes 600 - 999

Draw the state of the heap and the free list after the following operations are executed.

```
ptr1 = malloc(100);
free(ptr1);
ptr2 = malloc(200);
ptr3 = malloc(400);
free(ptr2);
ptr4 = malloc(300);
ptr5 = malloc(50);
```

Operation 6: `ptr4 = malloc(300);`



Free list

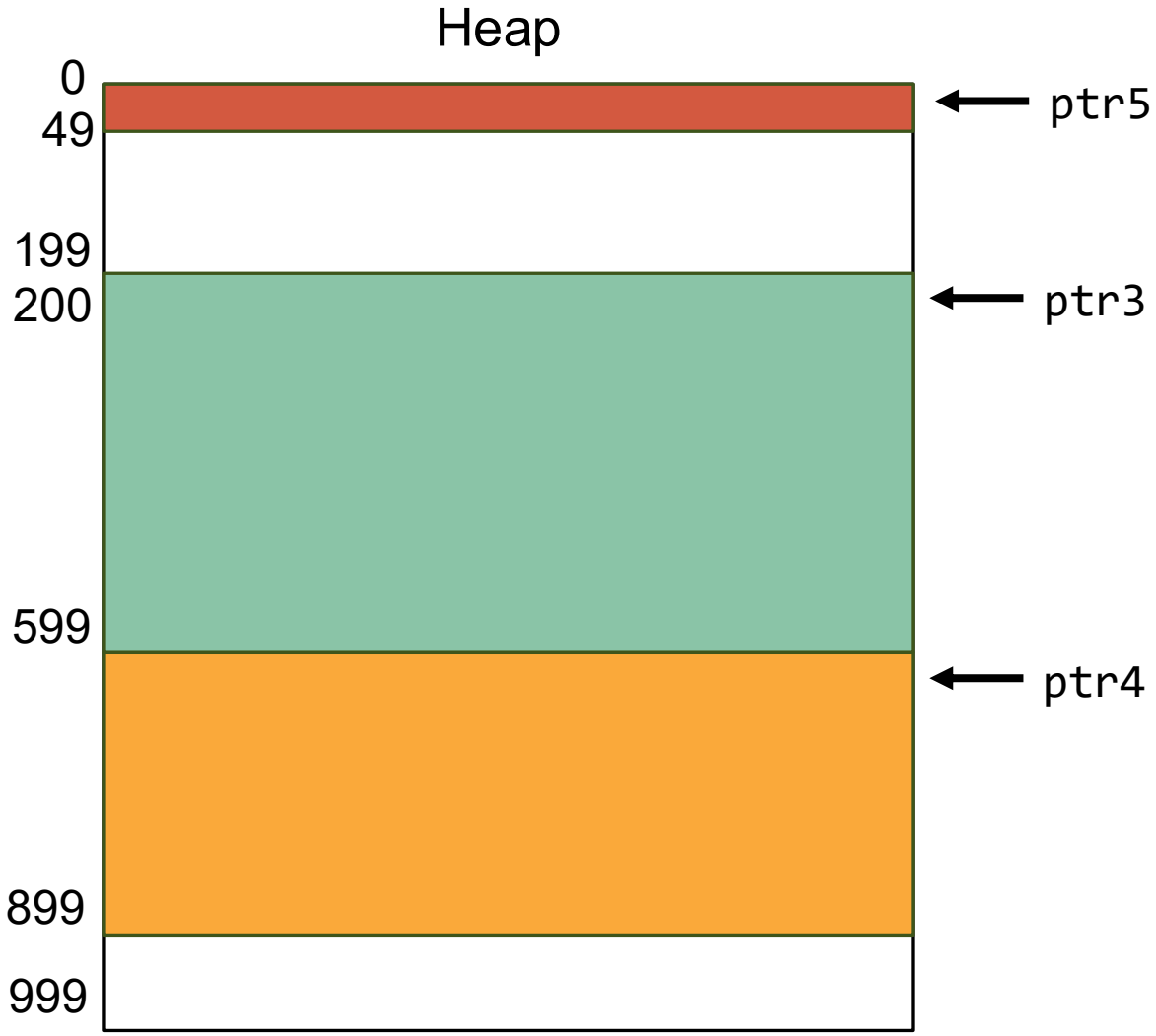
Bytes 0 - 199

Bytes 900 - 999

Draw the state of the heap and the free list after the following operations are executed.

```
ptr1 = malloc(100);
free(ptr1);
ptr2 = malloc(200);
ptr3 = malloc(400);
free(ptr2);
ptr4 = malloc(300);
ptr5 = malloc(50);
```

Operation 7: `ptr5 = malloc(50);`



Free list

Bytes 50 - 199

Bytes 900 - 999

Draw the state of the heap and the free list after the following operations are executed.

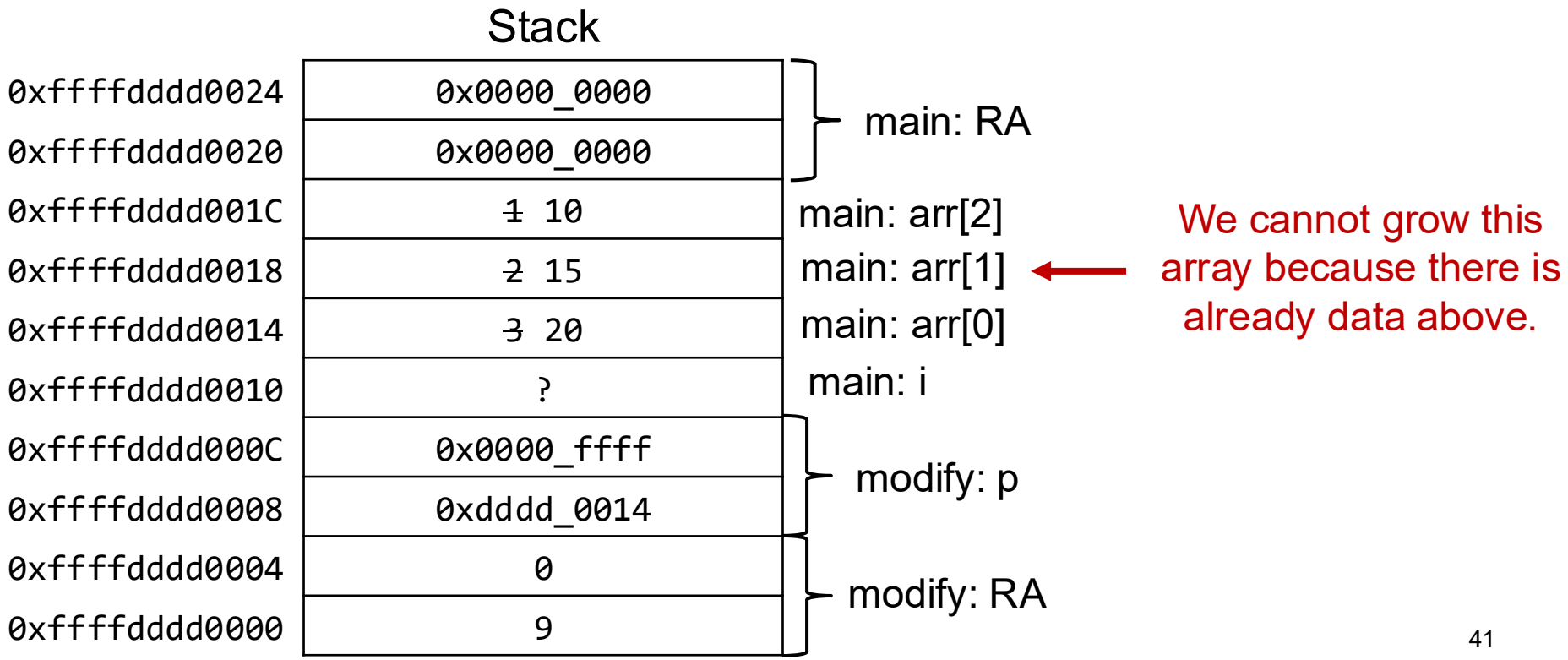
```
ptr1 = malloc(100);
free(ptr1);
ptr2 = malloc(200);
ptr3 = malloc(400);
free(ptr2);
ptr4 = malloc(300);
ptr5 = malloc(50);
```

Resizing Arrays

- Arrays are stored **contiguously**, meaning each element sits directly next to the one before it in memory.
- When an array is stored on the **stack**, its size is fixed and cannot be changed. This is because there may already be other data occupying the memory immediately after the array, leaving no room to expand it in place.

Resizing Arrays

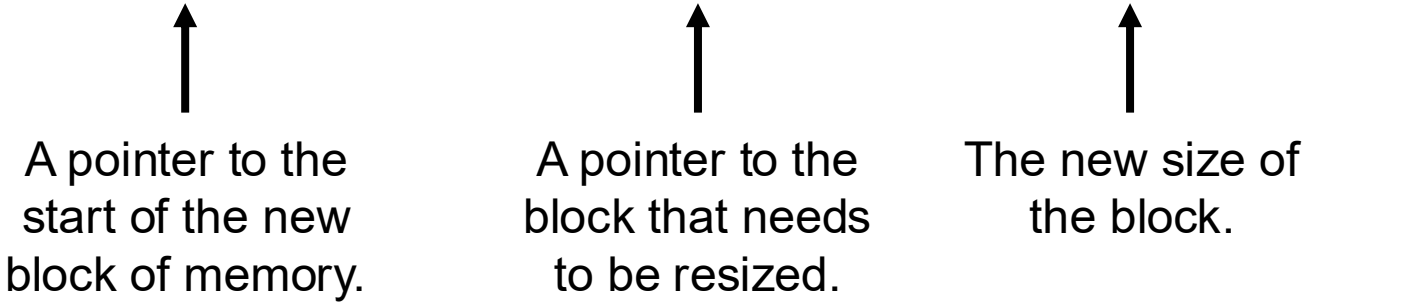
- Arrays are stored **contiguously**, meaning each element sits directly next to the one before it in memory.
- When an array is stored on the **stack**, its size is fixed and cannot be changed. This is because there may already be other data occupying the memory immediately after the array, leaving no room to expand it in place.



Realloc

- Stack arrays cannot be resized, but heap arrays can be resized with **realloc**
- **realloc** stands for reallocate. It's a function that changes the size of a block of memory previously allocated on the heap.

```
void *realloc(void *ptr, size_t new_size);
```



- Under the hood, **realloc** can do one of two things (covered on the next slides)

Case 1: Room to Grow in Place

- If there's space directly after the current memory block, **realloc** just expands the block in place.

Case 2: No room - must move

- If there's not enough room after the current block (maybe something else is in the way), **realloc** will:

1. Allocate a new block of the requested size.
2. Copy the contents from the old block to the new one.
3. Free the old block (if successful).
4. Return a pointer to the new block.

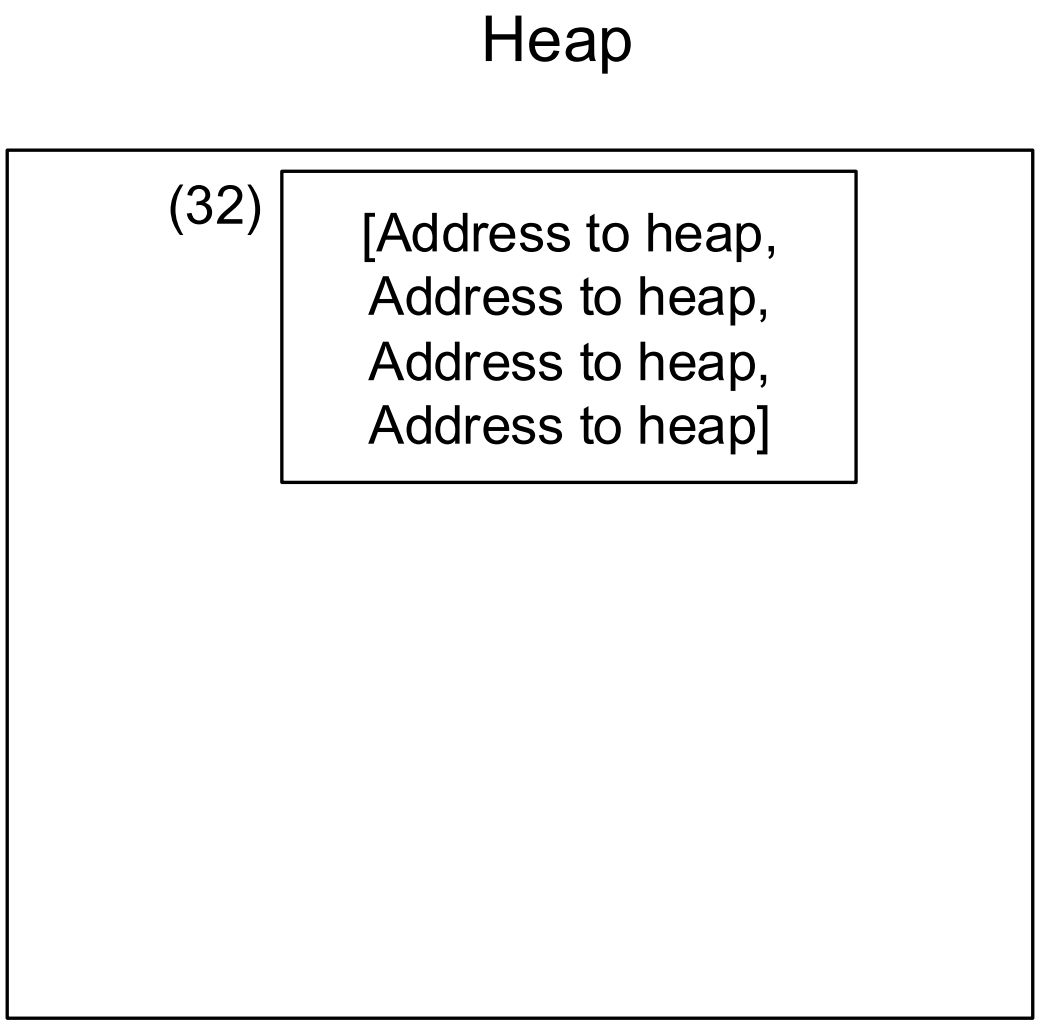
Arrays of Strings

Arrays of Strings

“Eliza”	“Aileen”	“Akshan”	“Avi”

Each string itself is an array of `char`

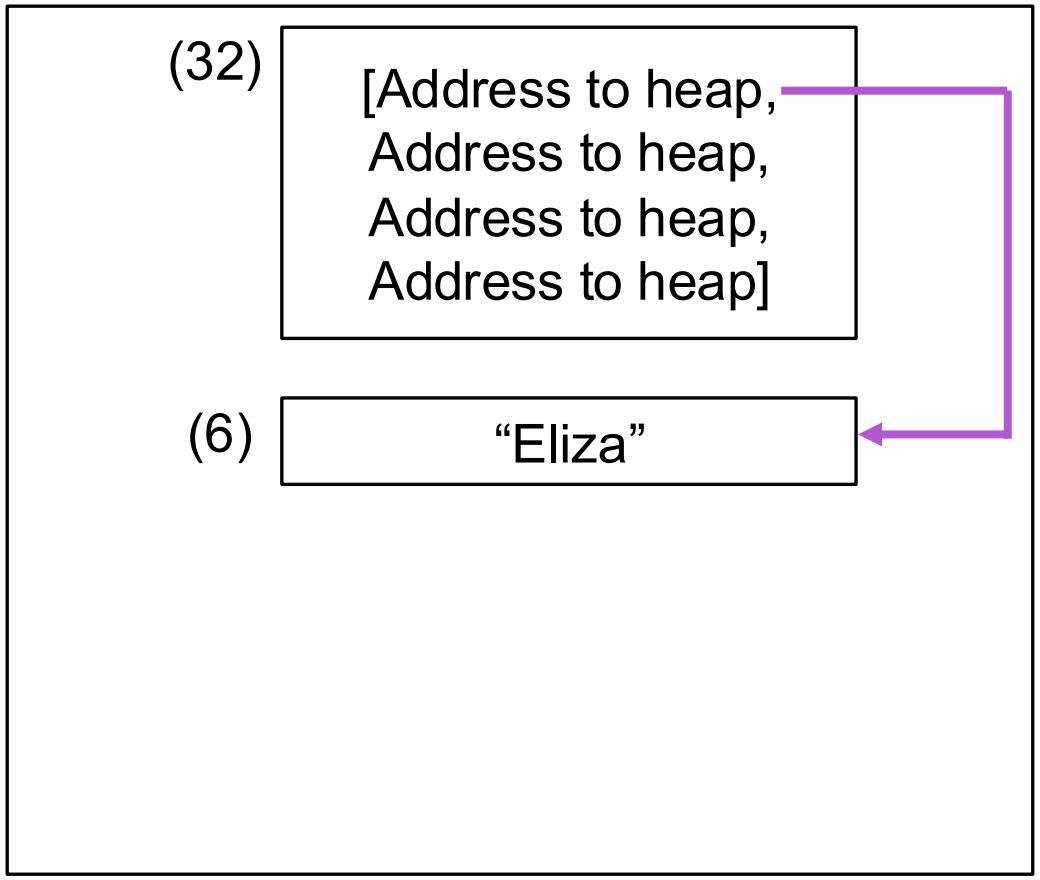
char** in Heap



The `char**` array occupies 32 bytes because it contains four pointers, and each pointer is 8 bytes (8 × 4 = 32).

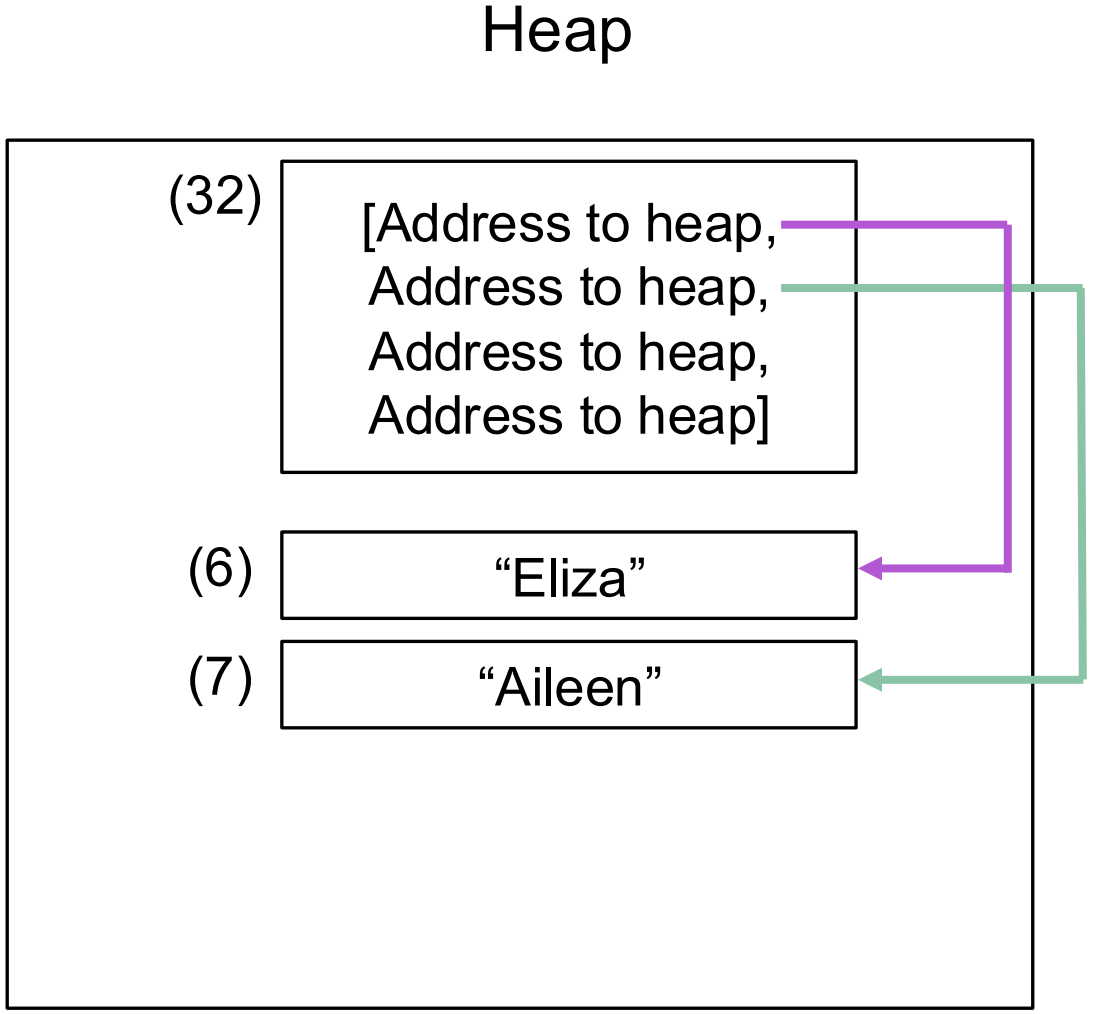
char** in Heap

Allocating space for the string `"Eliza"` requires 6 bytes: five bytes for the characters and one additional byte for the terminating null character.”

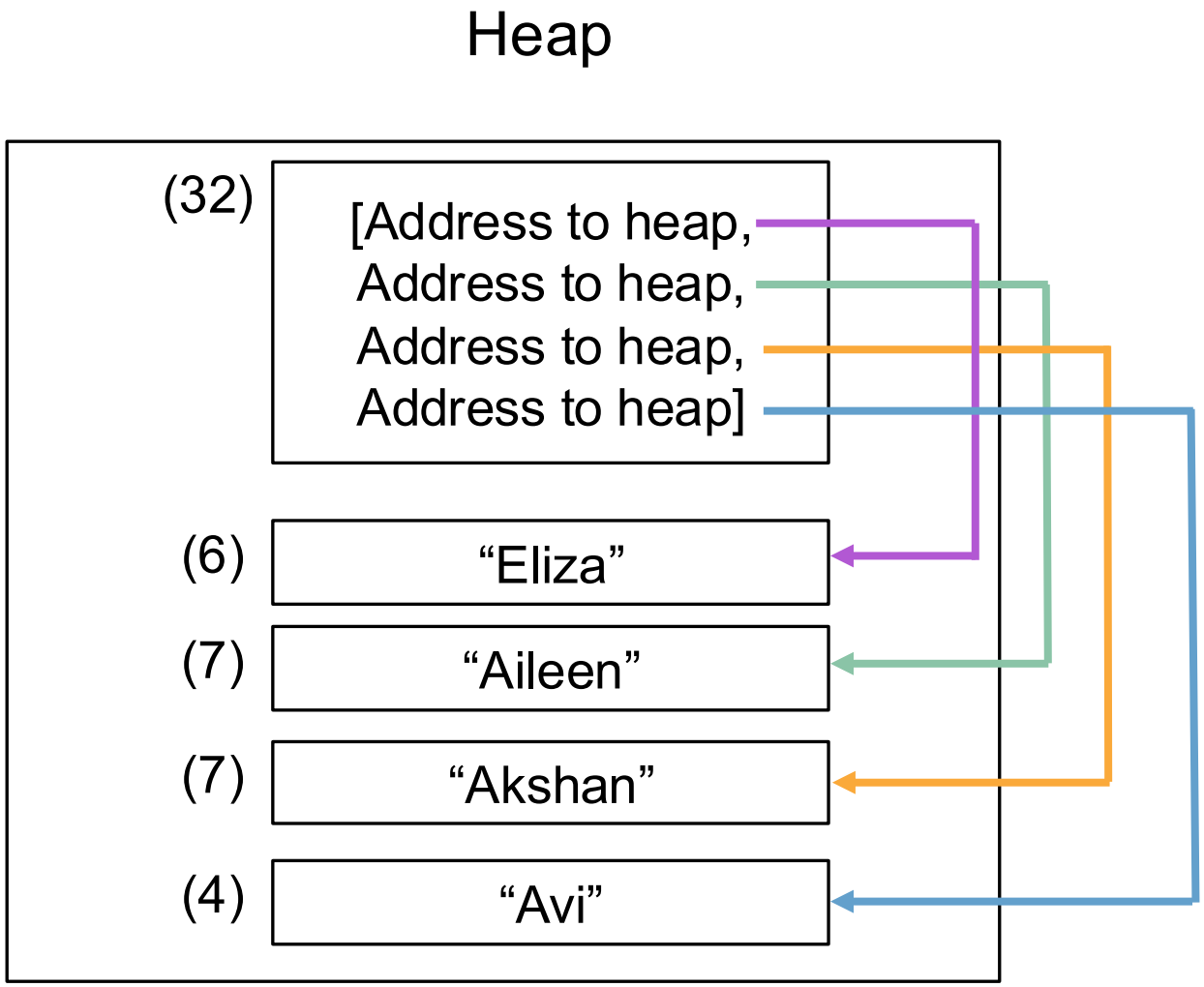


char** in Heap

Allocating space for the string `"Aileen"` requires 7 bytes: six bytes for the characters and one additional byte for the terminating null character.



char** in Heap


Heap
(32)
[Address to heap,
Address to heap,
Address to heap,
Address to heap]
(6)
“Eliza”
(7)
“Aileen”
(7)
“Akshan”
(4)
“Avi”

Buffers

- A buffer is just a chunk of memory used to store data — like an array.
- When you write something into a buffer, you're filling up that space in memory.

```
char name[5];
```

`name` is a **buffer** that can hold a string of up to 4 characters plus the null terminator.

fgets

- `fgets` reads a line of input from a given stream (`stdin` in this case) and stores it in the buffer you provide.
- The second argument (`sizeof(buffer)`) tells `fgets` the maximum number of characters to read, including the null terminator (`\0`).
- If the input fits in the buffer, it's stored as a null-terminated string.
- If the input is longer than the buffer, only part of it is read, and the rest remains in the input stream to be consumed later.
- `fgets` returns the same pointer you passed in (`buffer`) on success, and `NULL` on failure (e.g., if end-of-file is reached or an error occurs).

read_names

- The function `read_names` dynamically allocates and fills a 2D array of characters (an array of strings) by reading a specified number of names from the user.
- `char** read_names(int num_names, int name_len);`
- Parameters
  - `num_names:` The number of names the function should read.
  - `name_len:` The maximum length allowed for each name (not including the null terminator).
- Return Value
  - Returns a `char**` (a pointer to an array of char*).
  - Each `char*` in the array points to a dynamically allocated string containing one name.
- The caller is responsible for freeing all allocated memory.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>


char** read_names(int num_names, int name_len) {

    char** names = malloc(                          );

    if (                                    ) {
        fprintf(stderr, "Memory allocation failed!\n");
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < num_names; i++) {

        names[i] = malloc(                          );

        if (                                ) {
            fprintf(stderr, "Memory allocation failed!\n");
            exit(EXIT_FAILURE);
        }
        printf("Enter name of student %d: ", i + 1);

        if (!fgets(names[i], name_len + 1, stdin)) {
            fprintf(stderr, "Error reading input!\n");
            exit(EXIT_FAILURE);
        }
    }

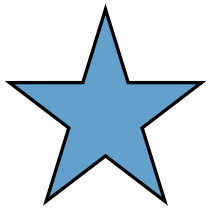
    return names;
}
```

How much memory should we allocate for the outer array?

How do we check if the allocation was successful?

How much memory should we allocate for each inner array?

How do we check if the allocation was successful?



```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>


char** read_names(int num_names, int name_len) {
    char** names = malloc(num_names * sizeof(char*));
    if (!names) {
        fprintf(stderr, "Memory allocation failed!\n");
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < num_names; i++) {
        names[i] = malloc((name_len + 1) * sizeof(char));
        if (!names[i]) {
            fprintf(stderr, "Memory allocation failed!\n");
            exit(EXIT_FAILURE);
        }
        printf("Enter name of student %d: ", i + 1);

        if (!fgets(names[i], name_len + 1, stdin)) {
            fprintf(stderr, "Error reading input!\n");
            exit(EXIT_FAILURE);
        }
    }

    return names;
}
```

array_of_strings.c

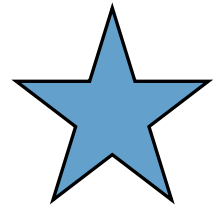
- This program builds a dynamically growing array of strings.
  - It repeatedly asks the user to enter words.
  - Each word is stored on the heap.
  - The program keeps the words in a dynamically resized array of char * pointers.
  - If the array fills up, it doubles its capacity using realloc.
  - When the user types "quit", the program:
    - Prints all stored words.
    - Frees all allocated memory.



```
/* TODO: Allocate memory for an array of char*
pointers with
INITIAL_CAPACITY elements. Each element will
eventually point to a
dynamically allocated string. */
words = malloc(capacity * sizeof(char*));
if (!words) {
   fprintf(stderr, "malloc failed");
   return 1;
}
```



```
/* TODO: Use realloc to resize the existing array of
char* pointers
to hold the new (doubled) capacity. Do NOT lose the
original
pointer if realloc fails. */
char** temp = realloc(words, capacity * sizeof(char*));
if (!temp) {
   fprintf(stderr, "realloc failed");
   break;
}
words = temp;
```



```
/* TODO: Allocate enough space on the heap to
store the buffer string in
the heap, including space for the null
terminator ('\0'). */
words[count] = malloc((strlen(buffer) + 1) *
sizeof(char));
if (!words[count]) {
   fprintf(stderr, "malloc failed");
   break;
}
```



```
printf("\nYou entered:\n");
for (size_t i = 0; i < count; i++) {
   printf("%zu: %s\n", i, words[i]);
   /* TODO: Free the dynamically allocated
   string
   to prevent a memory leak. */
   free(words[i]);
}

/* TODO: After freeing each individual
string, free the array
that stores the char* pointers. */
free(words);
```